Project Title:Tic_Tac_Toe with Display

Semester Project: Digital Logic Design

Course: BSCS-15C

Teacher:Engr Arshad Nazir

Submitted by:

Name	CMS_Id
Rohama Nazir	556025
Hareem Umer	544975
Nabiha Iftikhar	542689
Fatima Tu Zahra	542329

Dedication

We dedicate this project to our parents, whose unconditional love, endless encouragement, and countless sacrifices have been the foundation of everything we have achieved. Their unwavering belief in us has carried us through every challenge of this academic journey.

We also dedicate this work to our teachers, who have consistently inspired us to think beyond theory and apply our knowledge to build something real and meaningful.

Acknowledgements

We would like to express our sincere gratitude to our course instructor, **Engr. Arshad Nazir**, and our lab instructor, **Mr. Mughees Ahmed**, for their invaluable guidance, patience, and constructive feedback throughout the course *Digital Logic Design*. Their support was instrumental in helping us translate a complex game concept into a fully functional hardware system.

We are also deeply thankful to the **School of Electrical Engineering and Computer Science (SEECS) at NUST** for providing access to the laboratory facilities, hardware components, and equipment necessary to simulate, build, and test our design.

Finally, we acknowledge the dedication and cooperative spirit of our project group — **Hareem Umer , Nabiha Iftikhar, Rohama Nazir, and Fatima tu Zahra** — whose collective effort, shared problem-solving, and teamwork made the successful completion of this project possible.

Abstract

This report presents the complete design and implementation of the classic Tic-Tac-Toe game using fundamental principles of Digital Logic Design (DLD). The game is implemented as a hardware circuit using D flip-flops, logic gates, bi-color LEDs, and push-buttons arranged in a 3×3 grid.

The circuit is organized into six functional blocks: the Input Block (which captures player moves via push-buttons and D flip-flops), the XOR Gate Block (which determines turn parity), the Control Block (which checks win conditions using Boolean expressions), the Terminator Block (which implements the anti-cheat system), and the 3×3 LED Grid (which displays the game state) and Counter Block.

A key feature of the design is an anti-cheat mechanism: once a cell is marked, repeated pressing of that button has no effect. The game also freezes automatically as soon as a player wins, and a dedicated draw indicator activates when all nine cells are filled without a winner. The entire circuit was simulated using Proteus Design Suite.Moreover,the design of the circuit includes a counter with display that shows best of three score for both the players.

List of Figures:

Table of Contents

Contents

Chapter 1: Introduction

1.1 Overview of Project

Tic-Tac-Toe, also known as Noughts and Crosses, is a pencil-and-paper game for two players that involves marking up a 3×3 matrix. Each turn sees one of the players mark a box; Player X goes first followed by Player O, with the objective being the creation of a sequence of three boxes horizontally, vertically, or diagonally. When all nine boxes are occupied but neither has won, the result is a tie.

In this project, a Tic-Tac-Toe game is constructed using only combinational/sequential digital circuits as per the rules of Digital Logic Design. In other words, instead of programming a computer or designing a software-based program, the circuit is created using only a set of D flip-flops, bi-color LEDs, push-buttons, and some gates (AND, OR, NOT, XOR).

Inputs to the design comprise nine push-buttons arranged in a 3×3 matrix. Outputs are bi-color LEDs (with green indicating Player X's box and red Player O's box). Also included are LEDs that indicate whether there has been a win/draw condition.

Key design challenges addressed in this project include:

- Distinguishing which player's turn is active (XOR parity logic)
- Preventing a player from re-pressing an already-used cell (D flip-flop latch)
- Detecting all eight winning conditions simultaneously (Control Block Boolean logic)
- Freezing the game upon a win to prevent further input (Terminator Block anti-cheat)
- Detecting and signaling a draw when all nine cells are filled

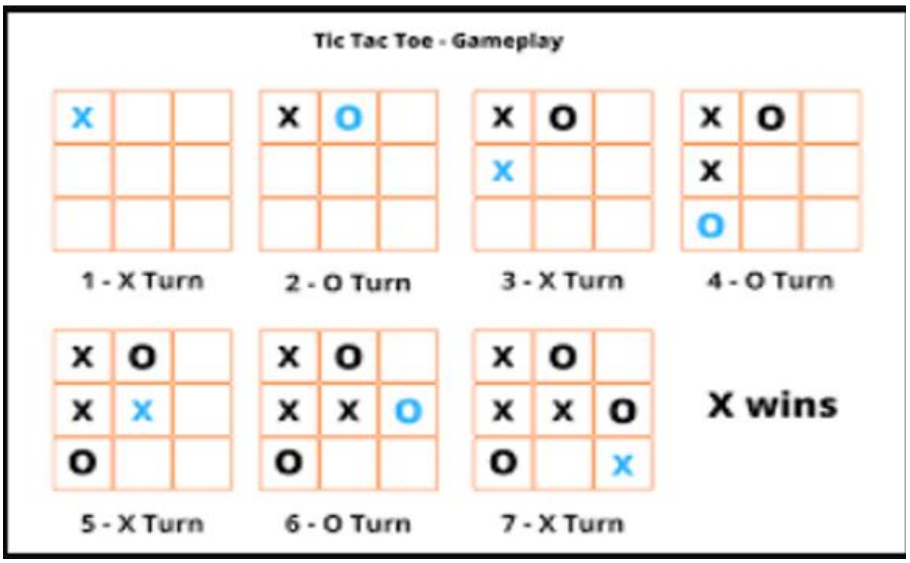


Figure 1: A sequence of moves by players O and X, leading to the inevitable winning of X..

1.2 Block Diagram of Complete System (without using ICs, just use simple blocks)

The six blocks are:

- Input Block: Nine push-buttons with D flip-flops to register and latch player moves.
- XOR Gate Block: Determines current player turn using XOR parity of total moves played.
- Control Block: Evaluates all win conditions using AND/OR Boolean functions.
- Terminator Block: Implements anti-cheat; freezes game and signals draw condition.
- 3×3 LED Grid: Displays moves — green for Player X, red for Player O.
- Counter Block :which displays the best of three scores for each player

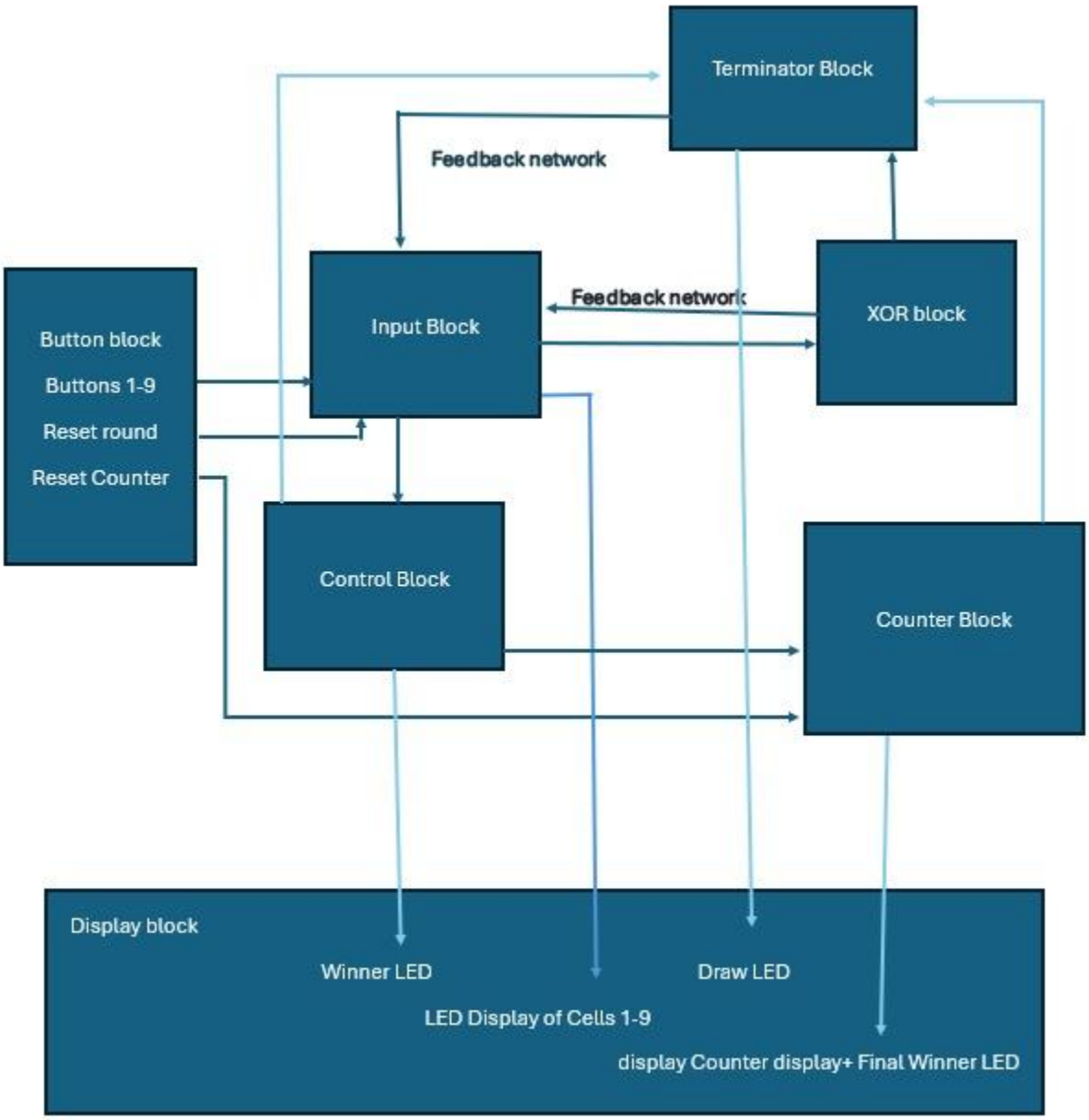


Figure 1: Block Diagram for Tic Tac Toe

1.3 Clear Work Division

Clearly describe the work of each member with the help of block diagram if possible. Or give a new figure illustrating work division

Team Member	Responsibilities
Rohama Nazir	**Control Block Boolean equations, Win condition AND/OR logic, Control LED , Report Writing**
Fatima Tu Zahra	**XOR Gate Block, turn parity logic, feedback network to Input Block , Report Writing**
Hareem Umer	**Input Block design, D flip-flop configuration , push-button debouncing, Counter Logic , Proteus Simulation and Design**
Nabiha Iftikhar	**Terminator Block anti-cheat system, Draw detection, Round+ Game End detection, Display and Buttons Block , Presentation Slides**

Figure 2: Work Division

Chapter 2: Design

2.1 Problem Statement.

Construct a complete two-player Tic-Tac-Toe game using only Digital Logic Design elements – D flip-flops, logic gates (AND, OR, NOT, XOR), push-buttons, and dual-colored LEDs. The design will:

- Read move selections from two users through a 3×3 matrix of push-buttons.
- Show move selections of the players using dual-color LEDs (Blue for player X and Red for player O).
- Determine whose turn it is by checking the parity of the moves made.
- Identify all eight win combinations (3 horizontal wins, 3 vertical wins, 2 diagonal wins).
- Prohibit any player from making a move on a cell that already contains one (anti-cheat latch).
- Lock the entire process when there's a win, making it impossible for anyone to continue playing.
- Identify the draw condition if all nine squares are occupied, without any player having won.
- Provide an option for Master Reset to start a new game at any point.
- Counter with display that determines best of three for both the players and then display the final winning player with indicated colored LED.

All logic is implemented using combinational and sequential digital circuits.

2.2 Truth Table / State Diagram

The game progresses through discrete states based on the number of moves played.

At any point, each cell is is one of three states: Empty (00), Occupied by X (10), or Occupied by O (01). The overall game state is the combination of all nine cell states.

The state transitions are governed by:

- If it is Player X's turn (odd move count) and an empty cell is pressed then that cell transitions to X-state (green LED on).
- If it is Player O's turn (even move count) and an empty cell is pressed then that cell transitions to O-state (red LED on).
- Once a cell is occupied, pressing its button has no effect — the D flip-flop A1 prevents re-triggering.
- If a player completes a winning line → game terminates, win LED activates, all inputs frozen.
- If all nine cells fill with no winner then draw LED activates

XOR Output Q (Player Turn)	DA2 (Input)	DA3 (Input)	LED Color
1 (Player X's Turn)	1	0	Green (X)
0 (Player O's Turn)	0	1	Red (O)

Table 1: Cell Flip-Flop Input/Output for Player Turn Determination

2.3 XOR Gate Block Turn Detection Logic

The XOR Gate Block determines which player is currently active. Since Player X always plays on odd turns (1st, 3rd, 5th, ...) and Player O on even turns (2nd, 4th, 6th, ...), the parity of total moves played determines the current turn.

Nine individual XOR gates are each connected to the A1 and A2 flip-flop outputs of one push-button cell. The output Oi of XOR gate i becomes 1 when that cell has been pressed. All nine outputs are chained through XOR gates to produce a final parity bit Q:

$$Q = O_1 \oplus O_2 \oplus O_3 \oplus O_4 \oplus O_5 \oplus O_6 \oplus O_7 \oplus O_8 \oplus O_9$$

When Q = 1 (odd number of buttons pressed), it is Player X's turn. When Q = 0 (even number pressed), it is Player O's turn. Q feeds into the D inputs of the upper flip-flops (DA2 = Q) and Q' feeds into the lower flip-flops (DA3 = Q'), completing the feedback network.

The truth table for a two-input XOR gate is:

Input A	Input B	Output (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

Table 2: XOR Gate Truth Table

2.4 Boolean Equations and K-Map Simplification

The Control Block evaluates whether any player has achieved a winning combination. The nine push-button positions are labeled A1 through A9 as shown below:

1	2	3
4	5	6
7	8	9

Table 3: Grid Numbering for Tic-Tac-Toe Board

The winning Boolean function Q(A1, A2, ..., A9) evaluates to 1 if any of the eight winning lines are completed by the same player:

$$Q = A_1A_2A_3 + A_4A_5A_6 + A_7A_8A_9 + A_1A_4A_7 + A_2A_5A_8 + A_3A_6A_9 + A_1A_5A_9 + A_3A_5A_7$$

The terms correspond to: three horizontal rows, three vertical columns, and two diagonals. This function is implemented separately for each player for Player X using the upper flip-flop outputs (QA2), and for Player O using the lower flip-flop outputs (QA3).

Since this is a 9-variable function, standard K-map minimization is impractical. However, the structure of the winning conditions is already minimal — each product term represents exactly one winning line, and

no further simplification is possible without losing a valid win condition. The function is therefore implemented directly as a two-level AND-OR network.

Term	Boolean Expression	Winning Combination
T1	A1 · A2 · A3	Top row
T2	A4 · A5 · A6	Middle row
T3	A7 · A8 · A9	Bottom row
T4	A1 · A4 · A7	Left column
T5	A2 · A5 · A8	Middle column
T6	A3 · A6 · A9	Right column
T7	A1 · A5 · A9	Main diagonal (top-left to bottom-right)
T8	A3 · A5 · A7	Anti-diagonal (top-right to bottom-left)

Table 4: Win Condition Boolean Terms

2.5 Complete Logic Diagrams

The following subsections present the circuit diagrams for each functional block of the design.

2.5.1 Input Block

Each of the nine push-buttons is connected to an identical sub-circuit consisting of three D flip-flops (A1, A2, A3). Flip-flop A1 latches the button press and prevents re-pressing. A2 and A3 register which player made the move based on the XOR block output.

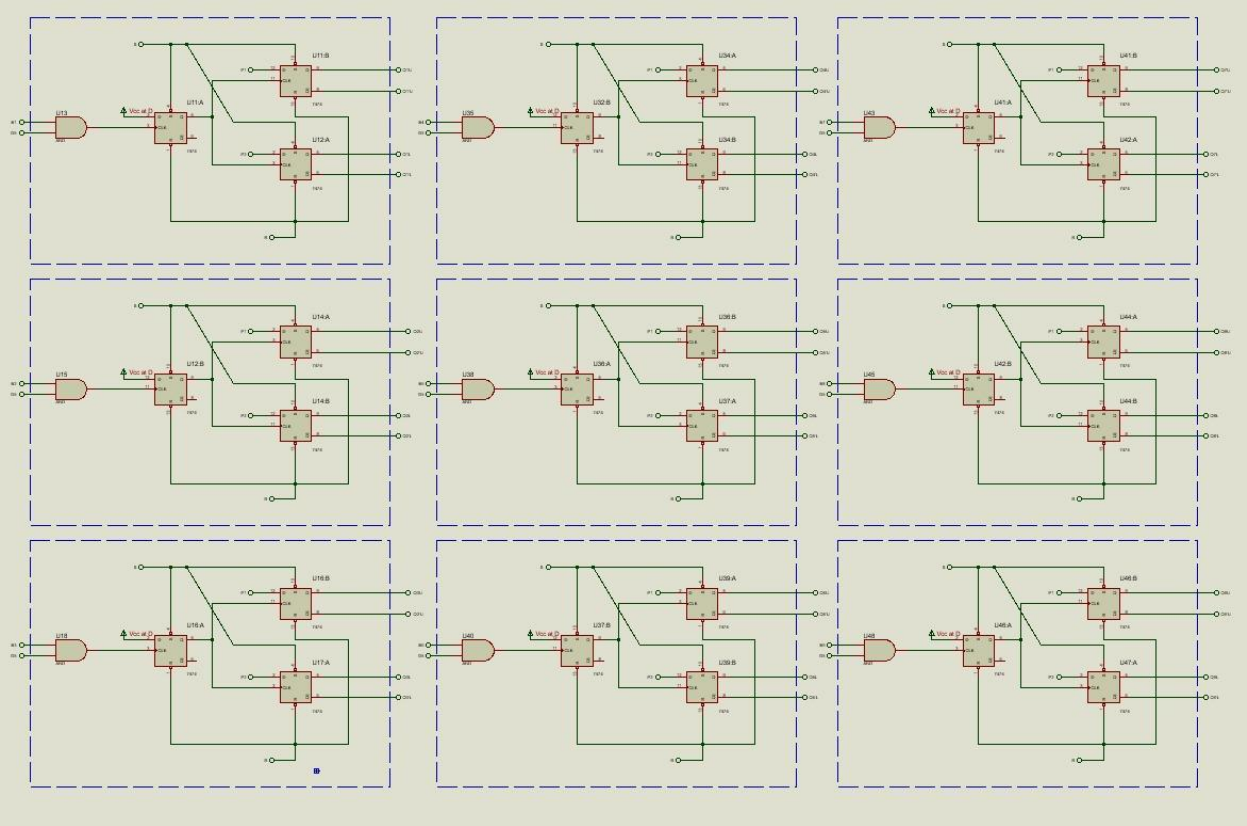


Figure 3.Input Block Circuit

2.5.1.1 Design of one cell (first cell)

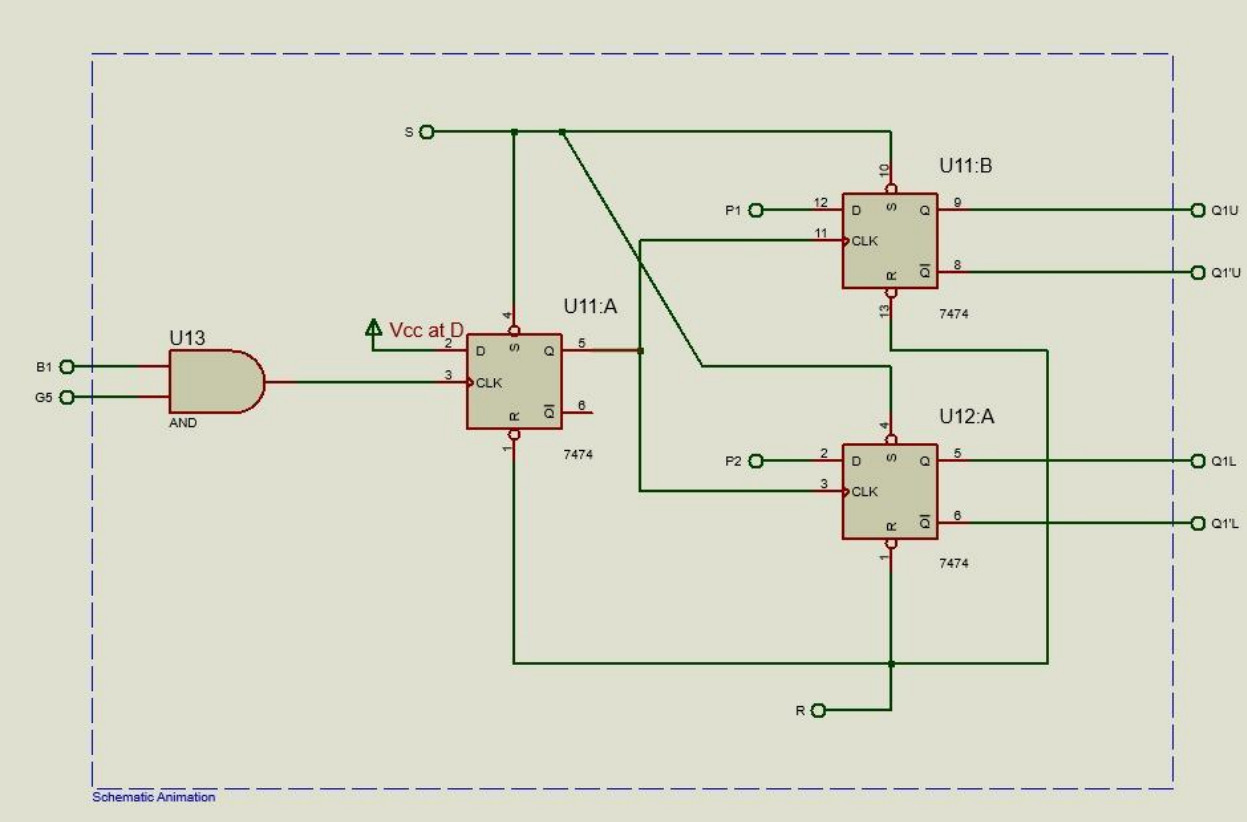


Figure 4 Single Push-Button Cell

2.5.2 XOR Gate Block

The XOR Gate Block processes the outputs of all nine cells and determines current player parity. The nine individual XOR gates feed into a chain of XOR gates producing the final Q output.

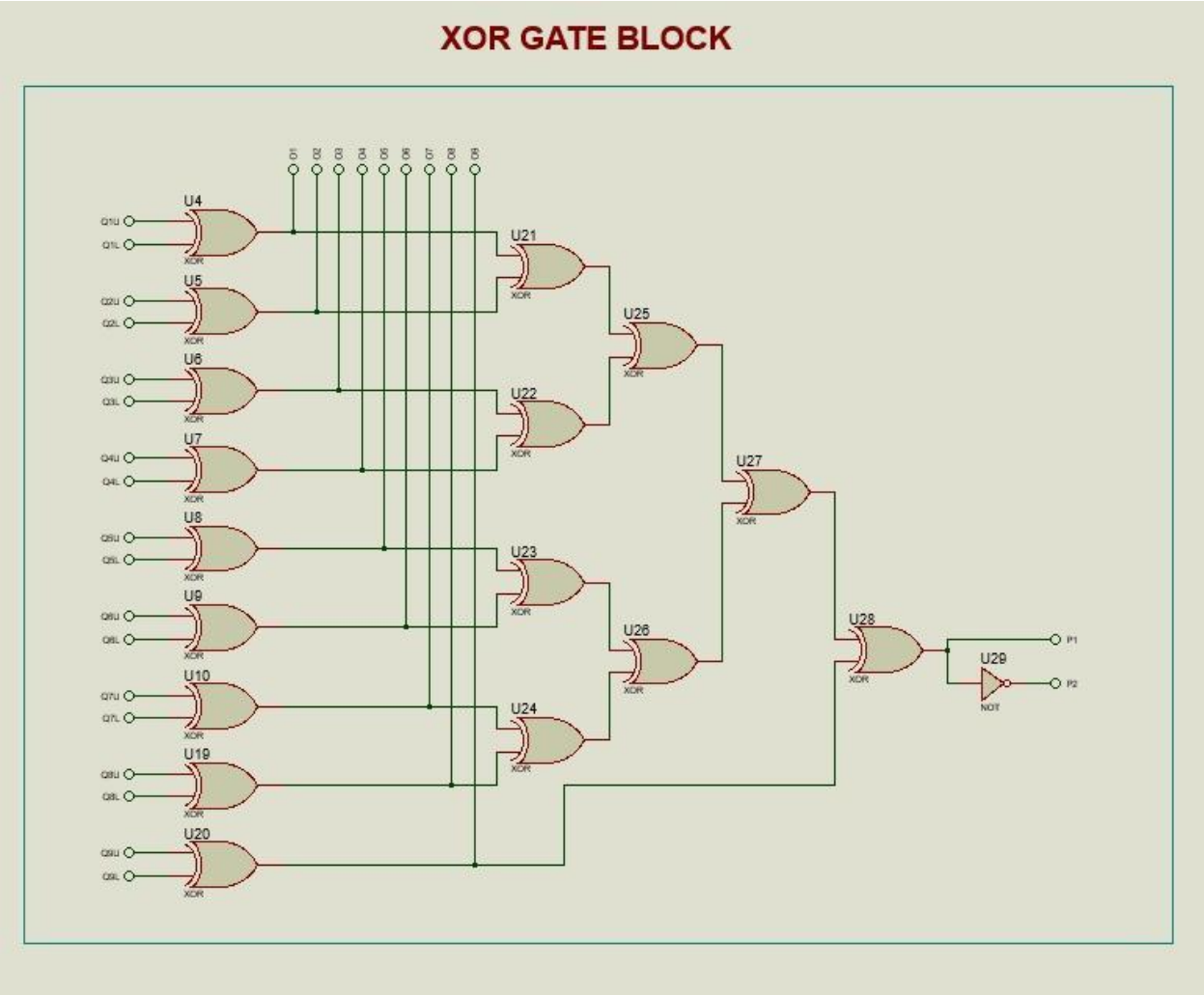


Figure 5: XOR Gate Block Circuit

2.5.3 Control Block

The Control Block implements the eight-term Boolean win function for each player. Eight 3-input AND gates feed into a single 8-input OR gate to produce QX (Player X win) and QO (Player O win).

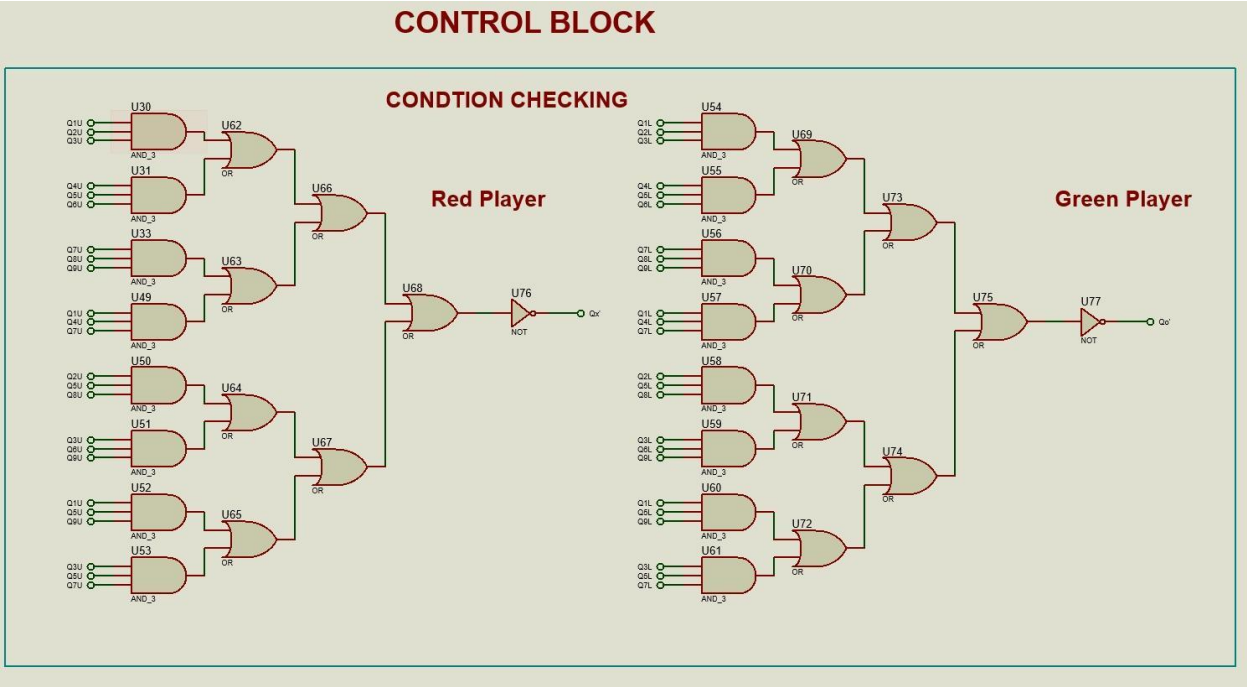


Figure 5: Control Block Circuit

2.5.3 Terminator Block

This block incorporates the anti-cheat system we wanted to implement in the circuit- after the game is won, further pressing of the push-buttons will have no effect on the output.

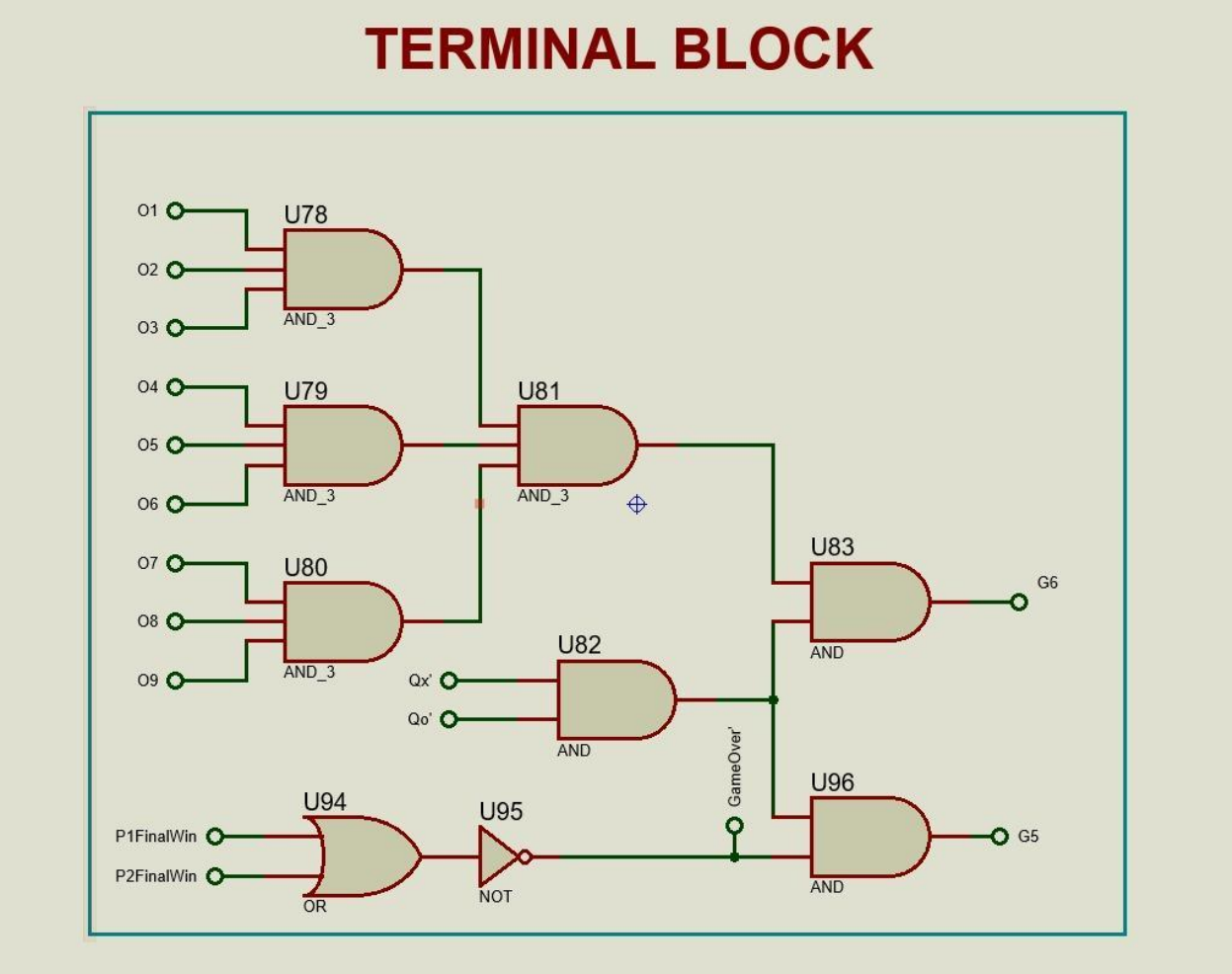
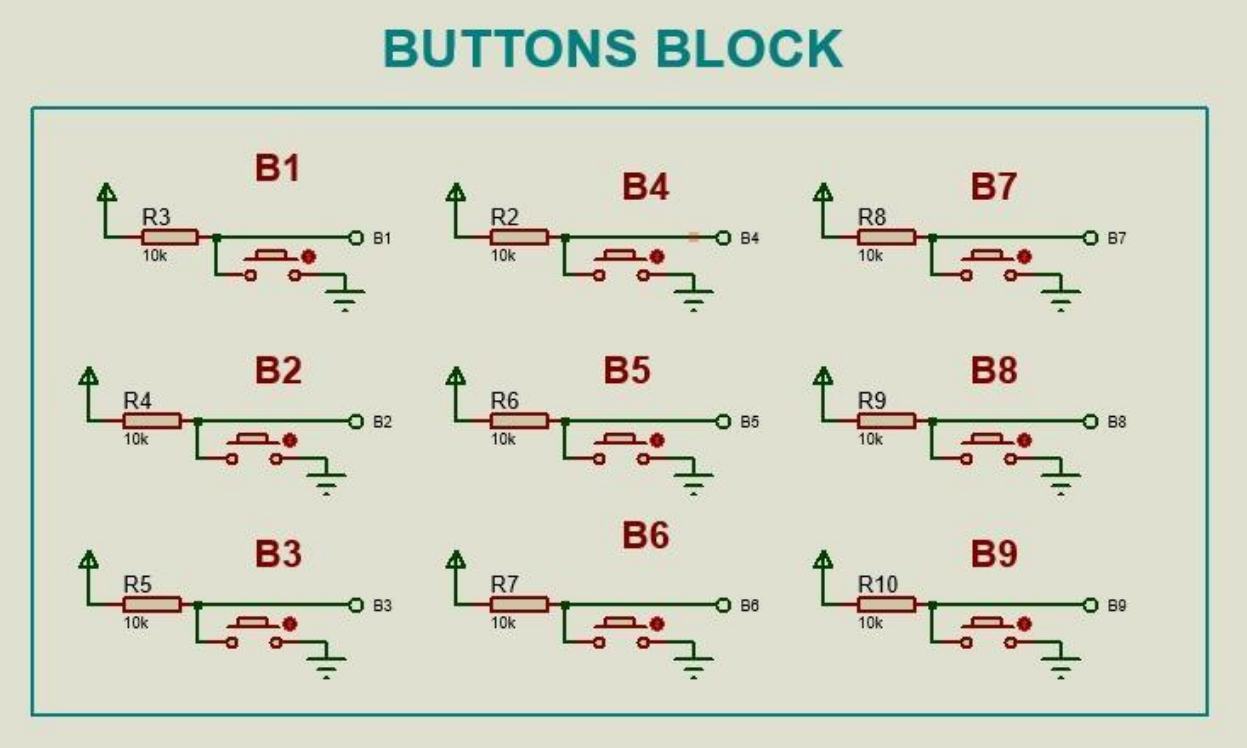


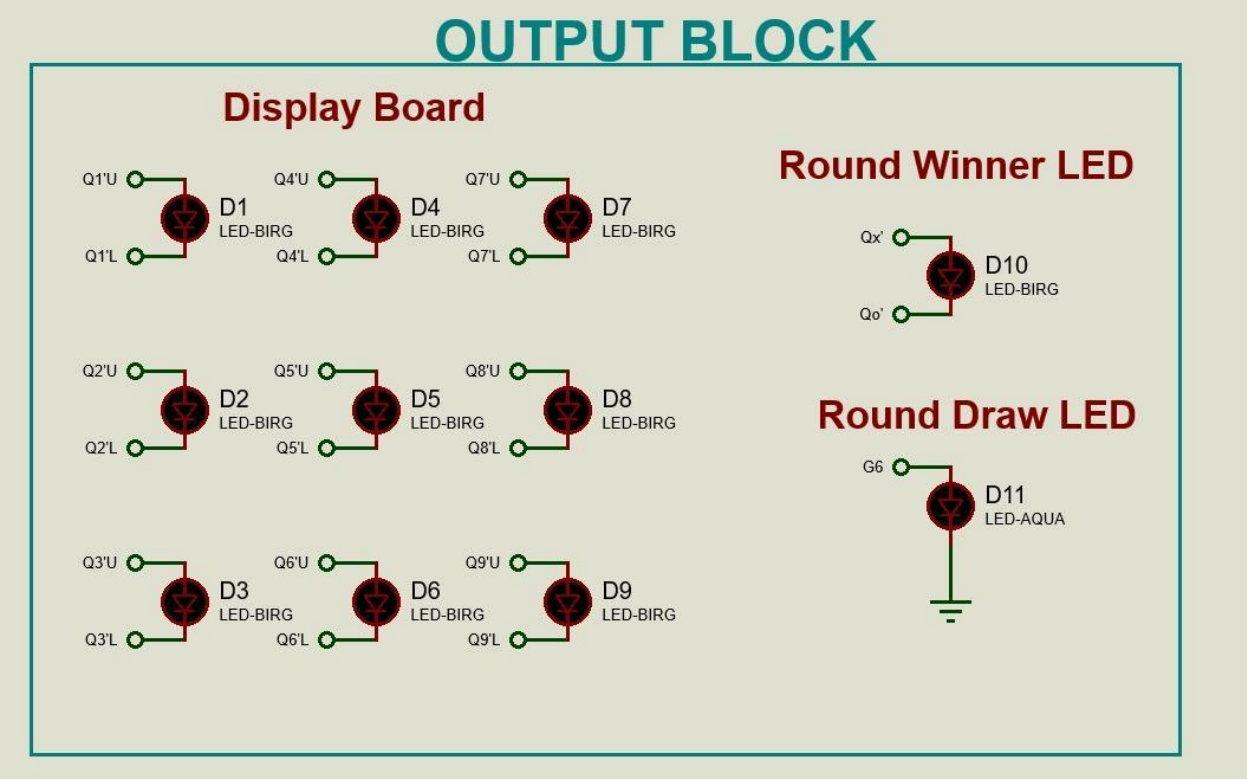
Figure 6:Terminator Block

2

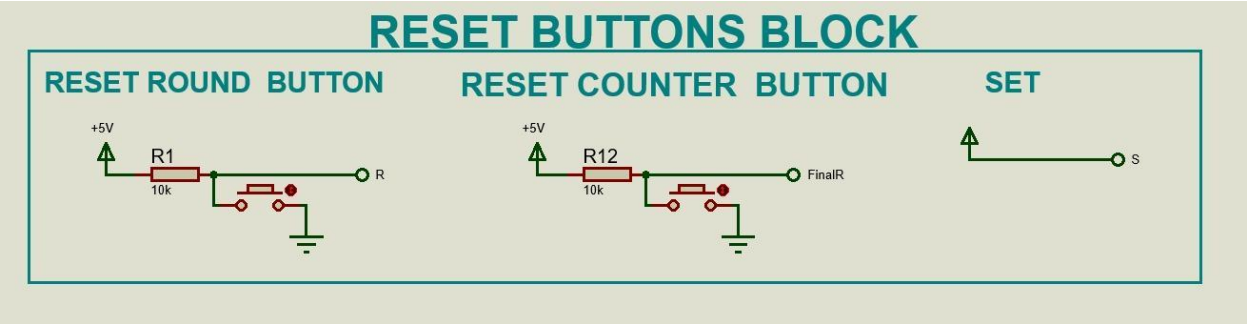
2.5.4 Button Block



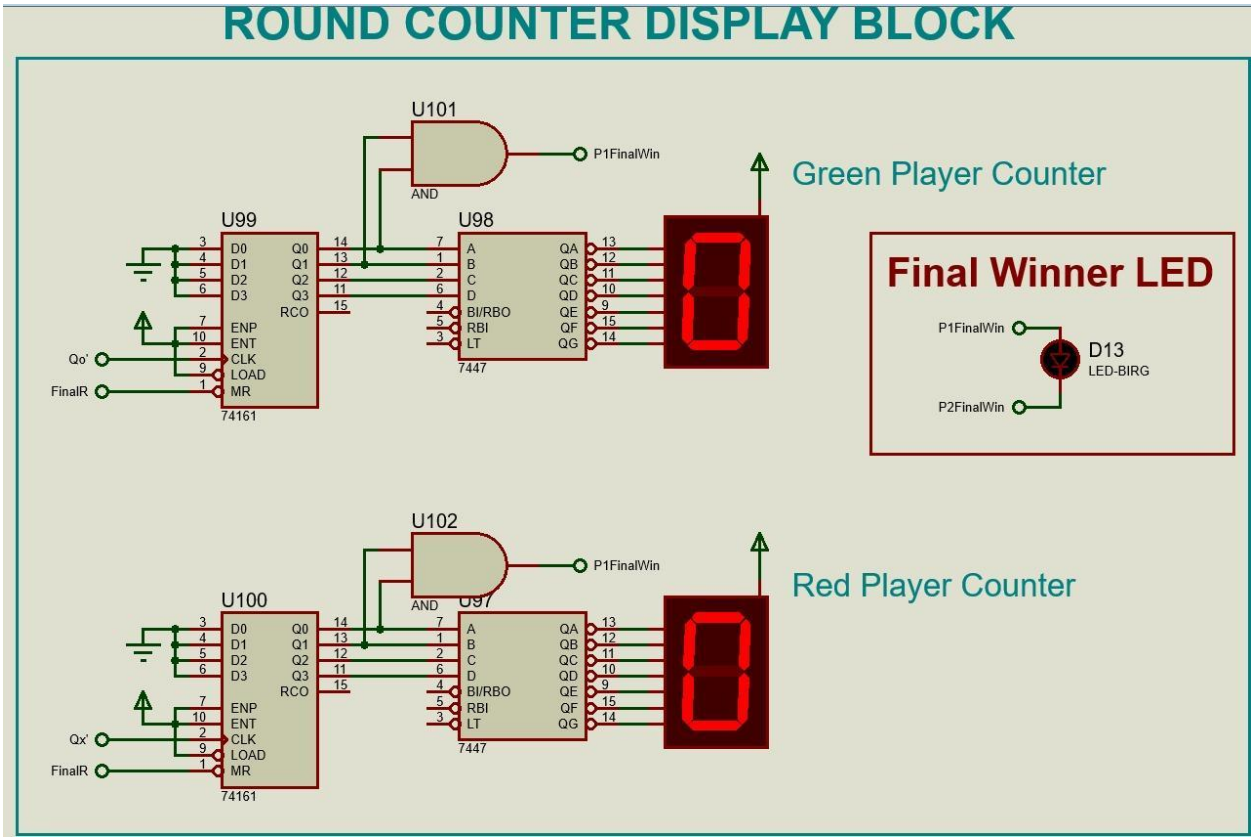
2.5.5 Output Block



2.5.6 Reset Button



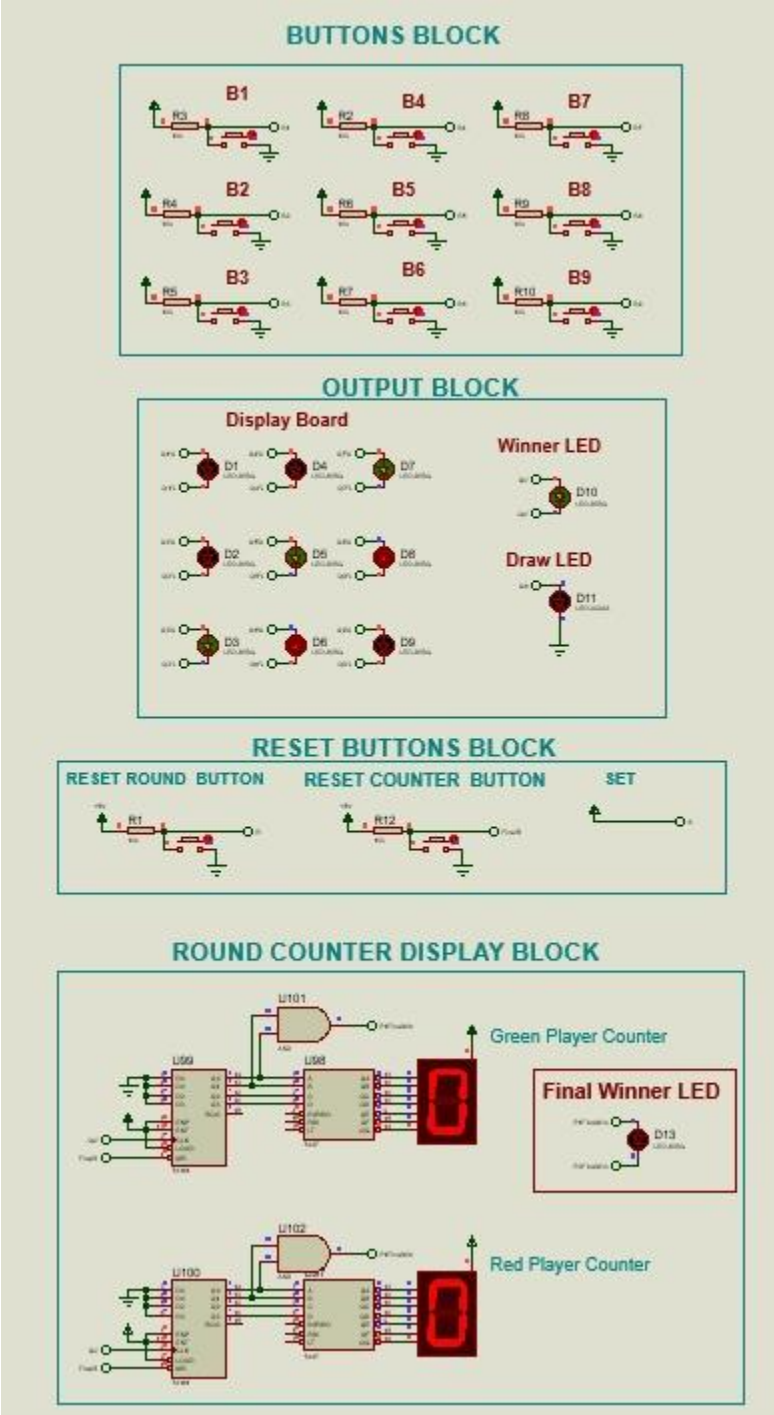
2.5.7 Counter Block



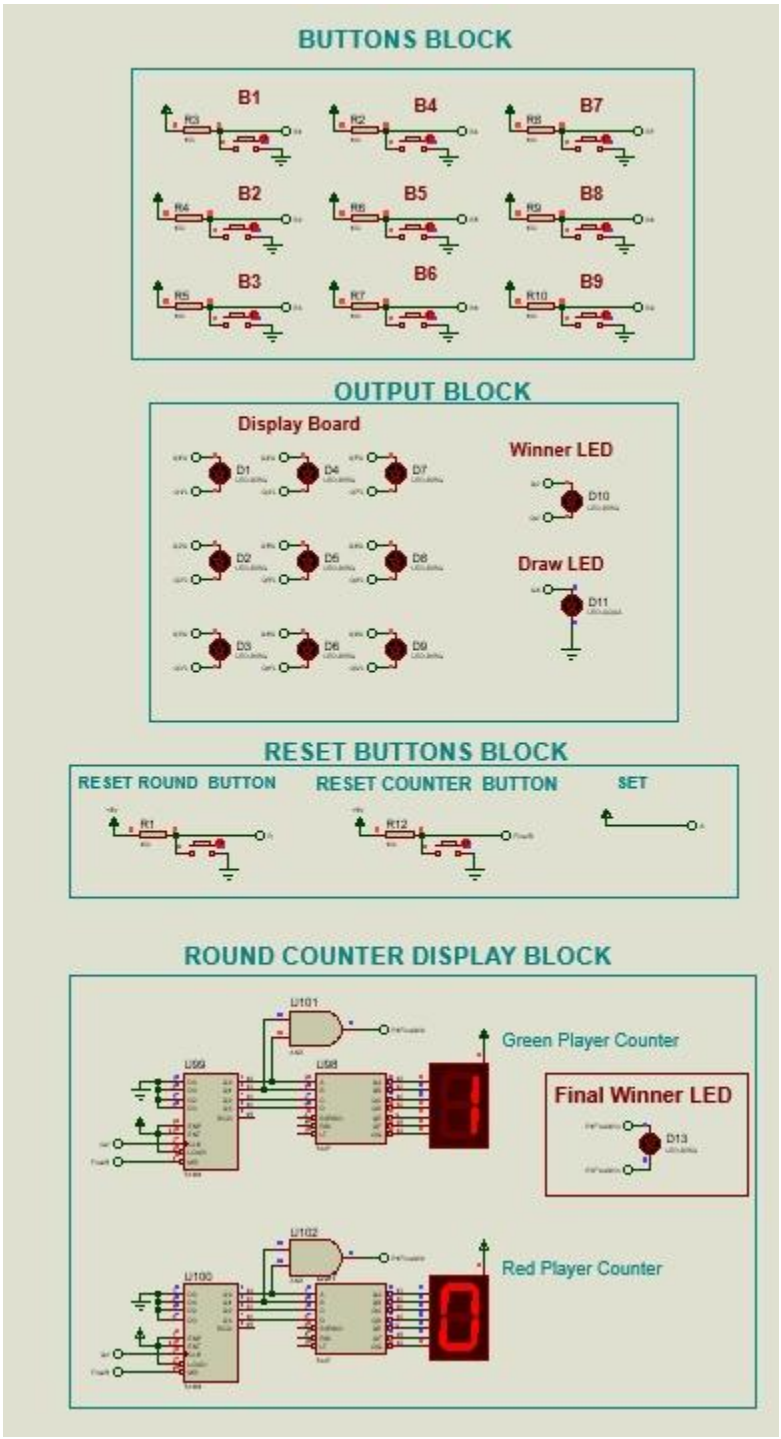
2.6 Proteus Simulation

2.6.1

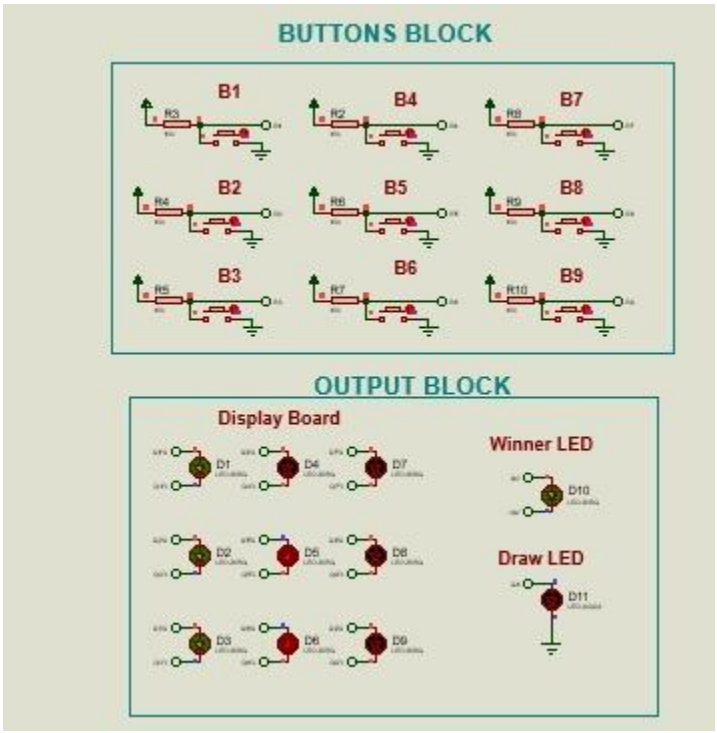
- Round 1 Green Player Wins



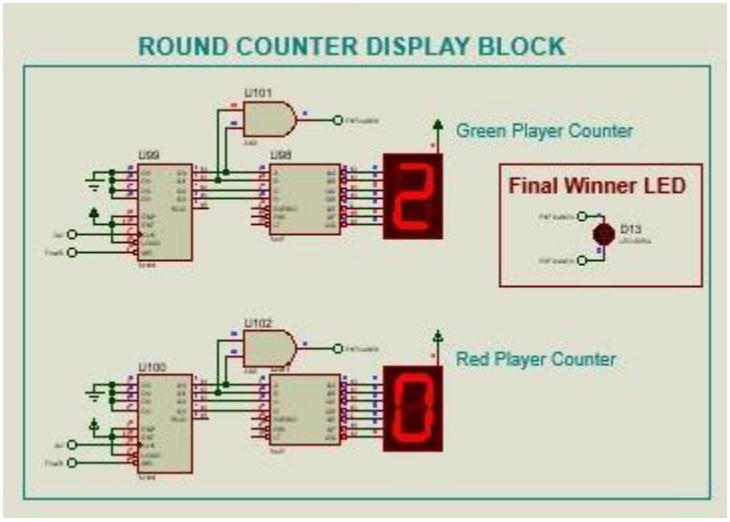
- Reset Round Button Pressed.Green Player Counter goes up by 1.



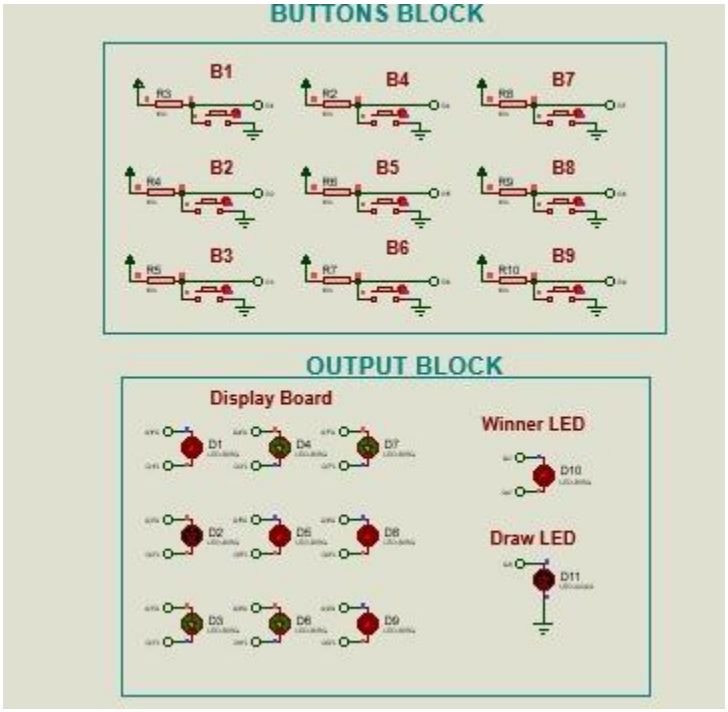
- Round 2 Green Player wins



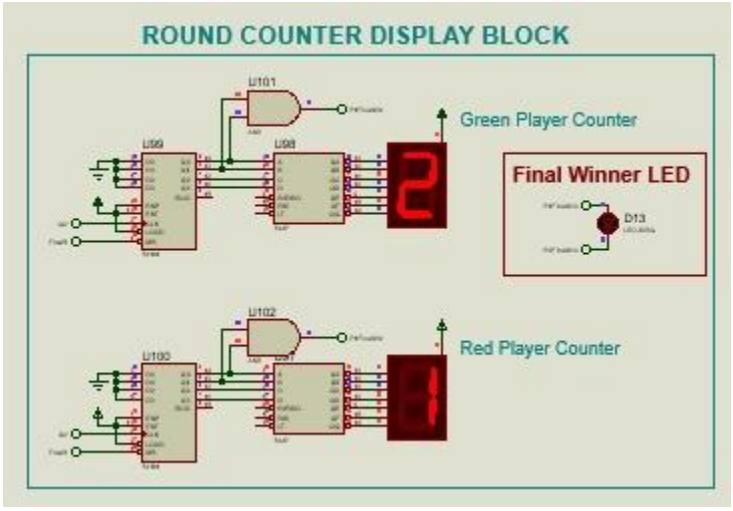
- Reset Round Button Pressed.Green Player Counter goes to 2



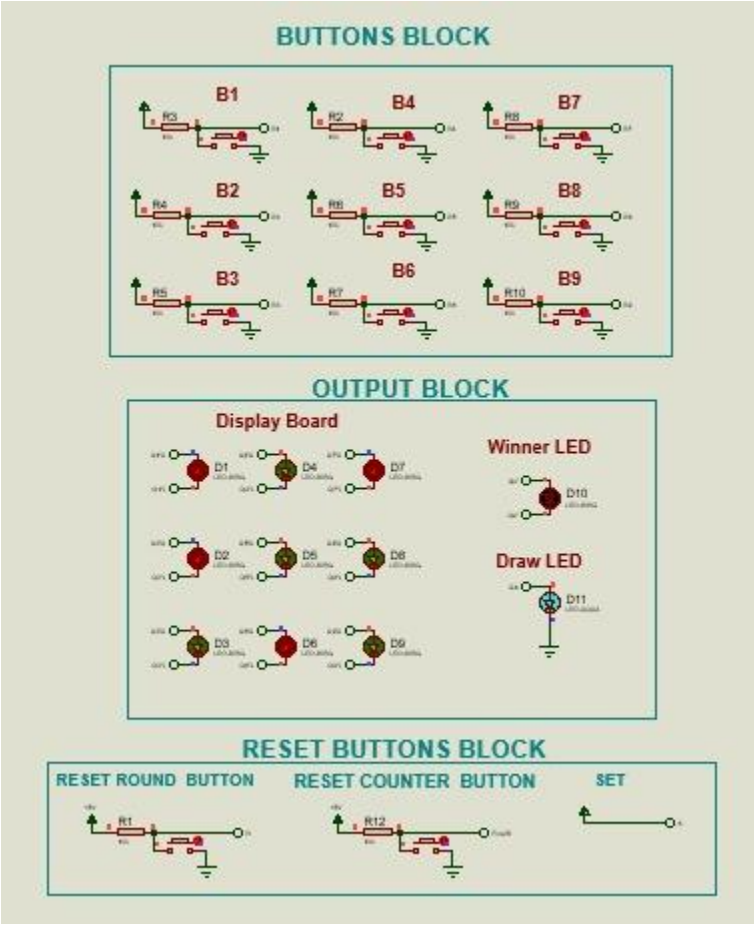
- Round 3 Red Player wins



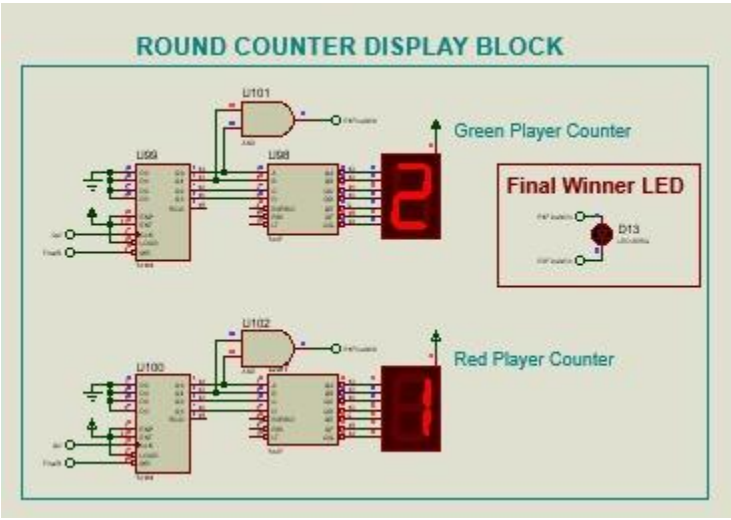
- Reset Round Button Pressed.Red Player Counter goes up by 1



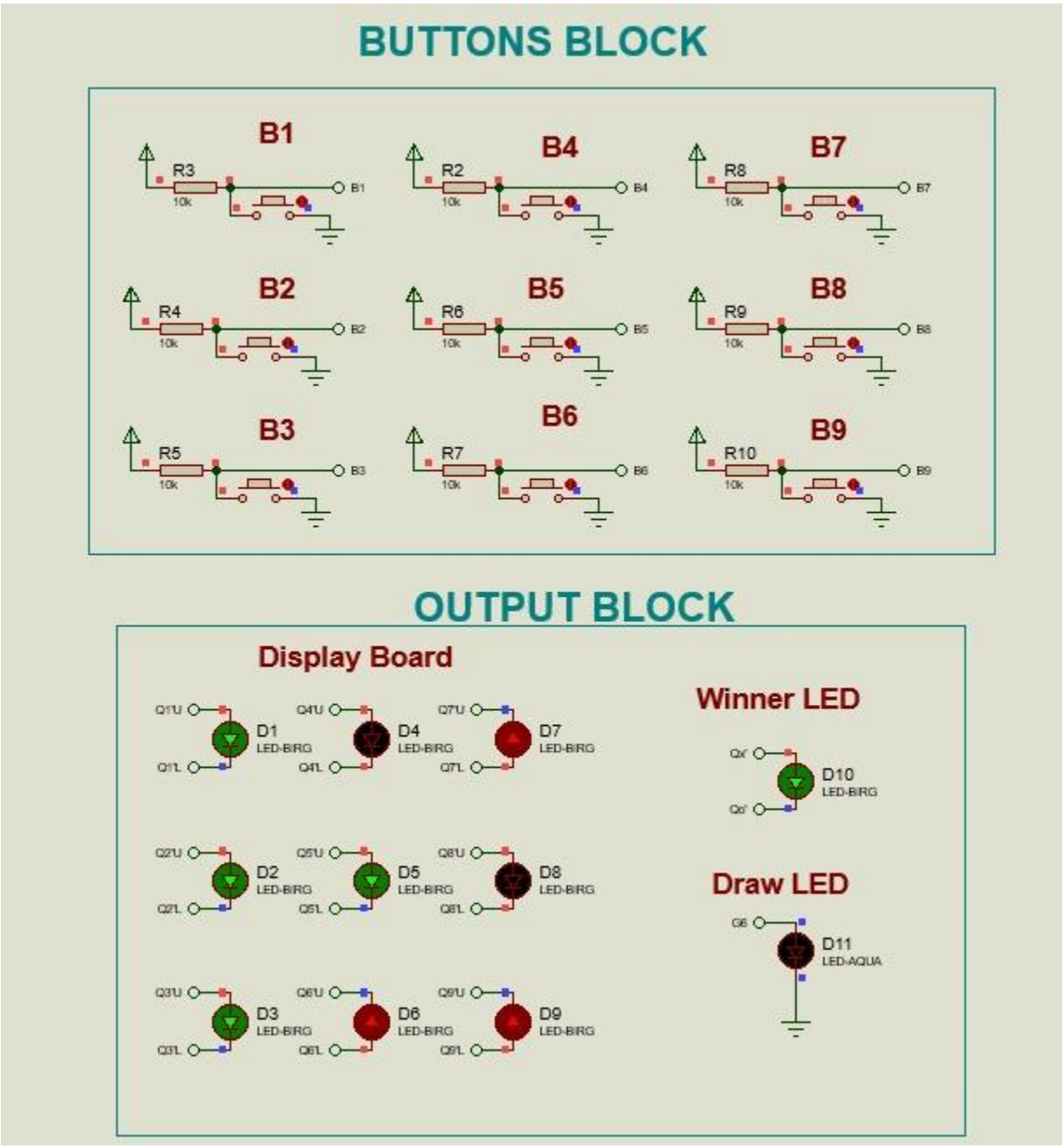
- Round 4 Draw



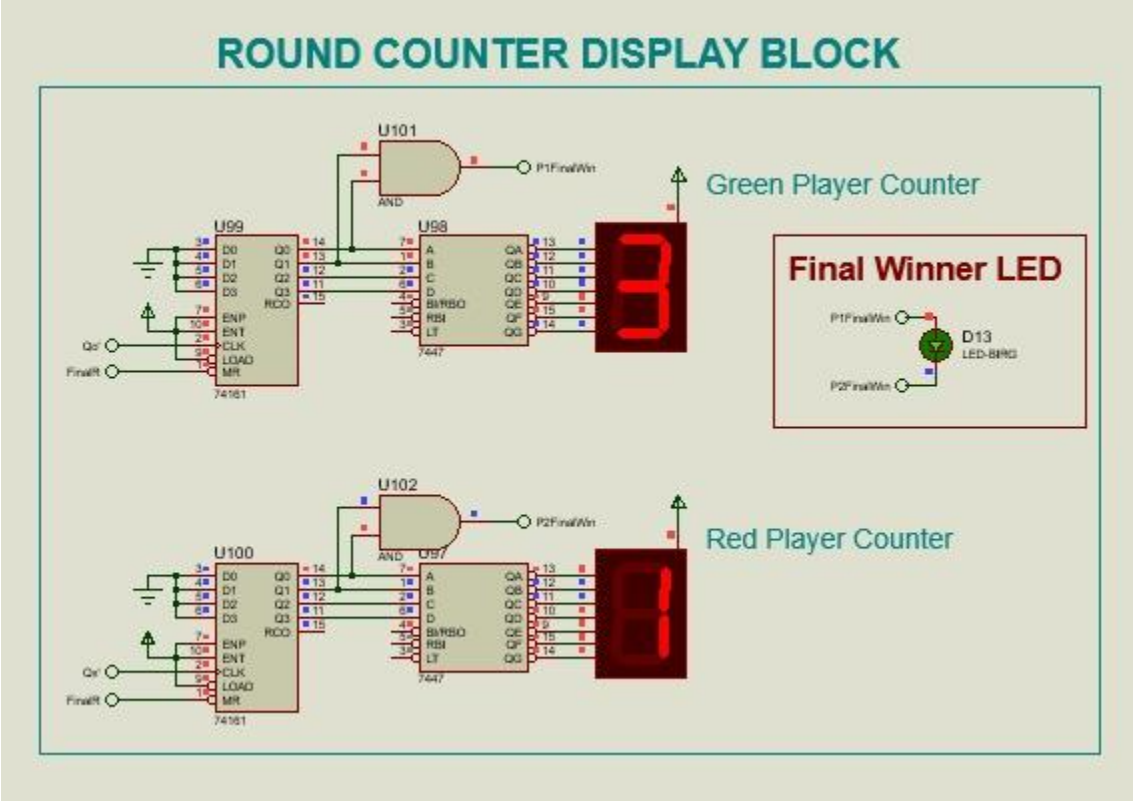
- Counter stay as they are



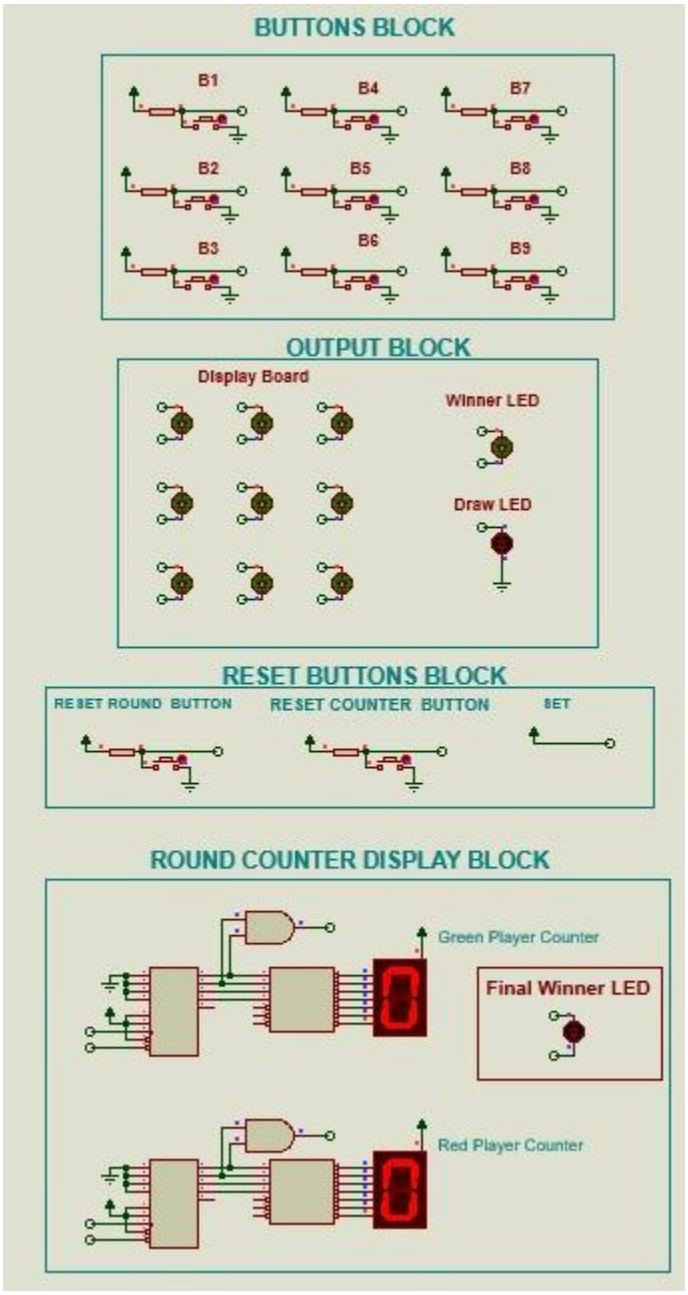
- Round 5 Green Player Wins



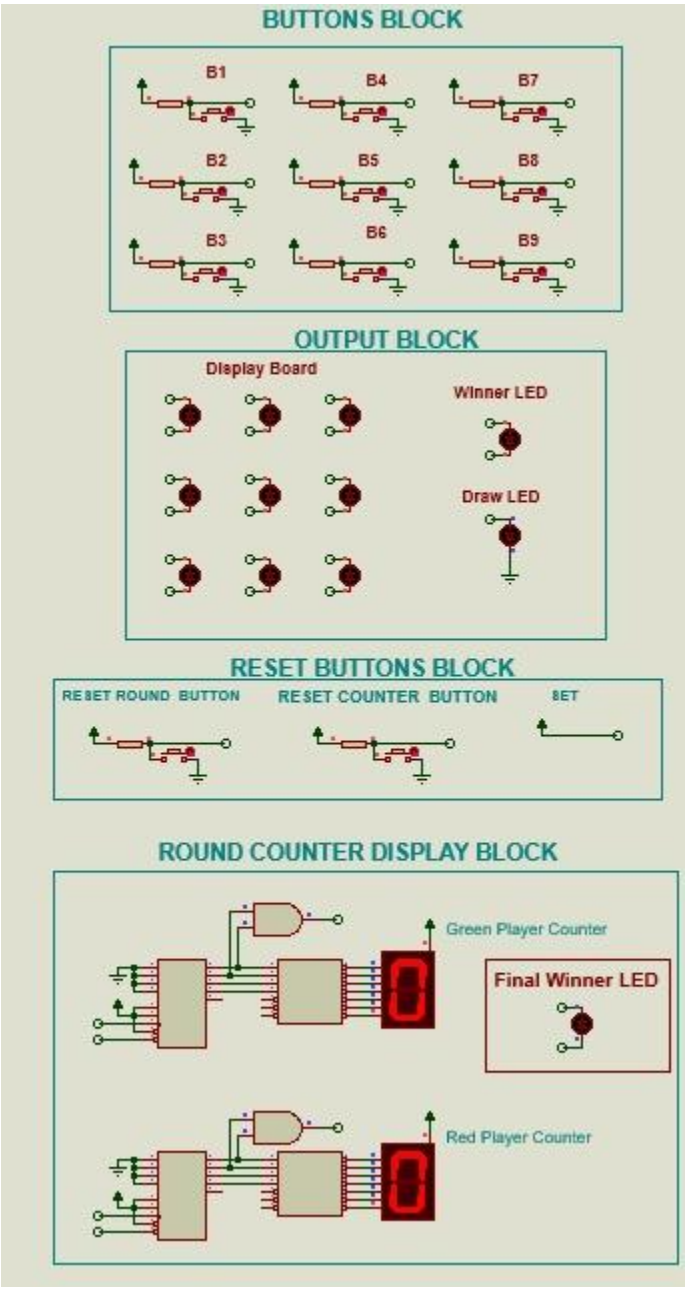
- Reset Round Button Pressed.Green Player Counter goes to 3.Final Win Led Turns on



- Counter Reset pressed



- Reset Counter>Reset Round >Reset Counter Game can be played again

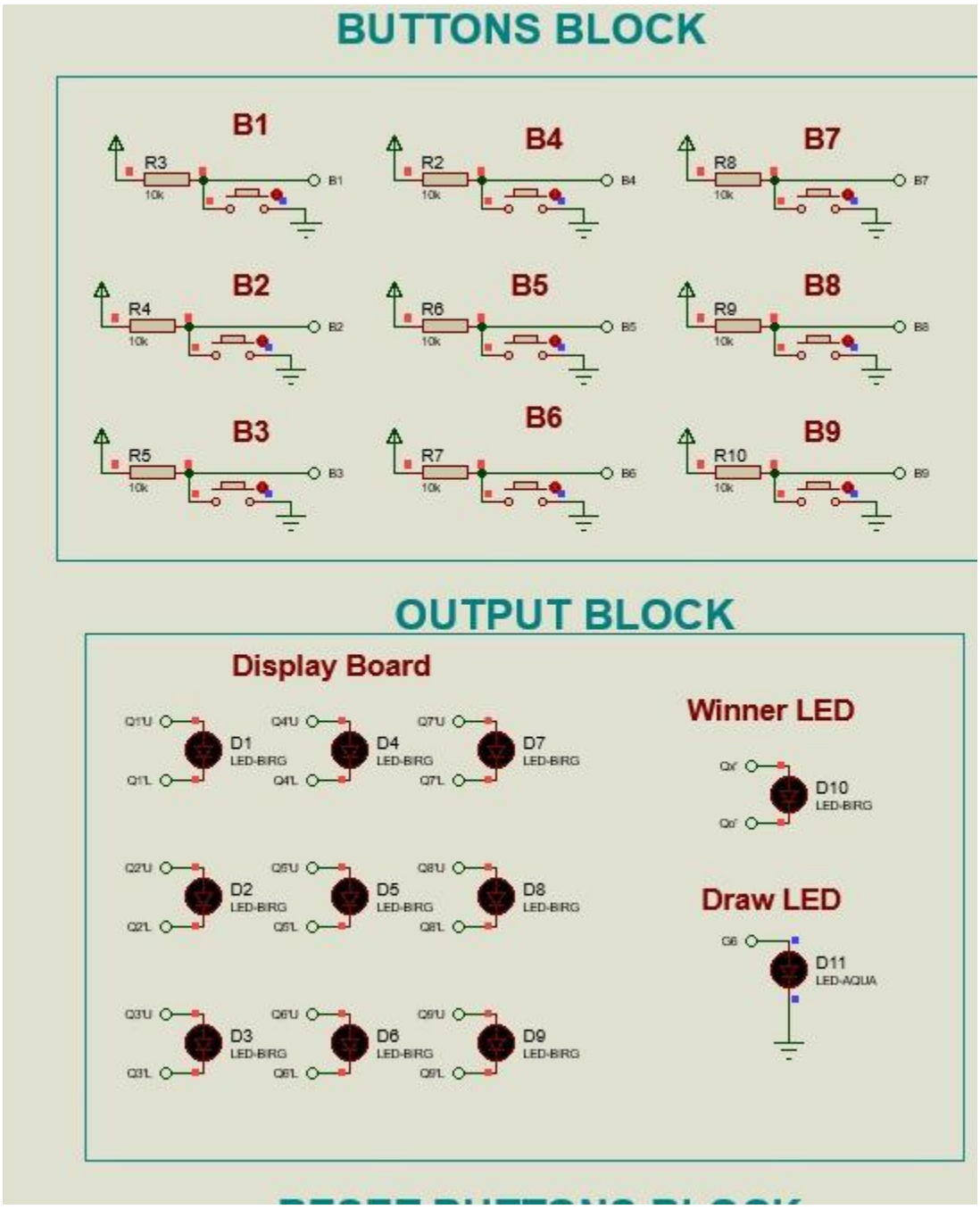


2.6.2

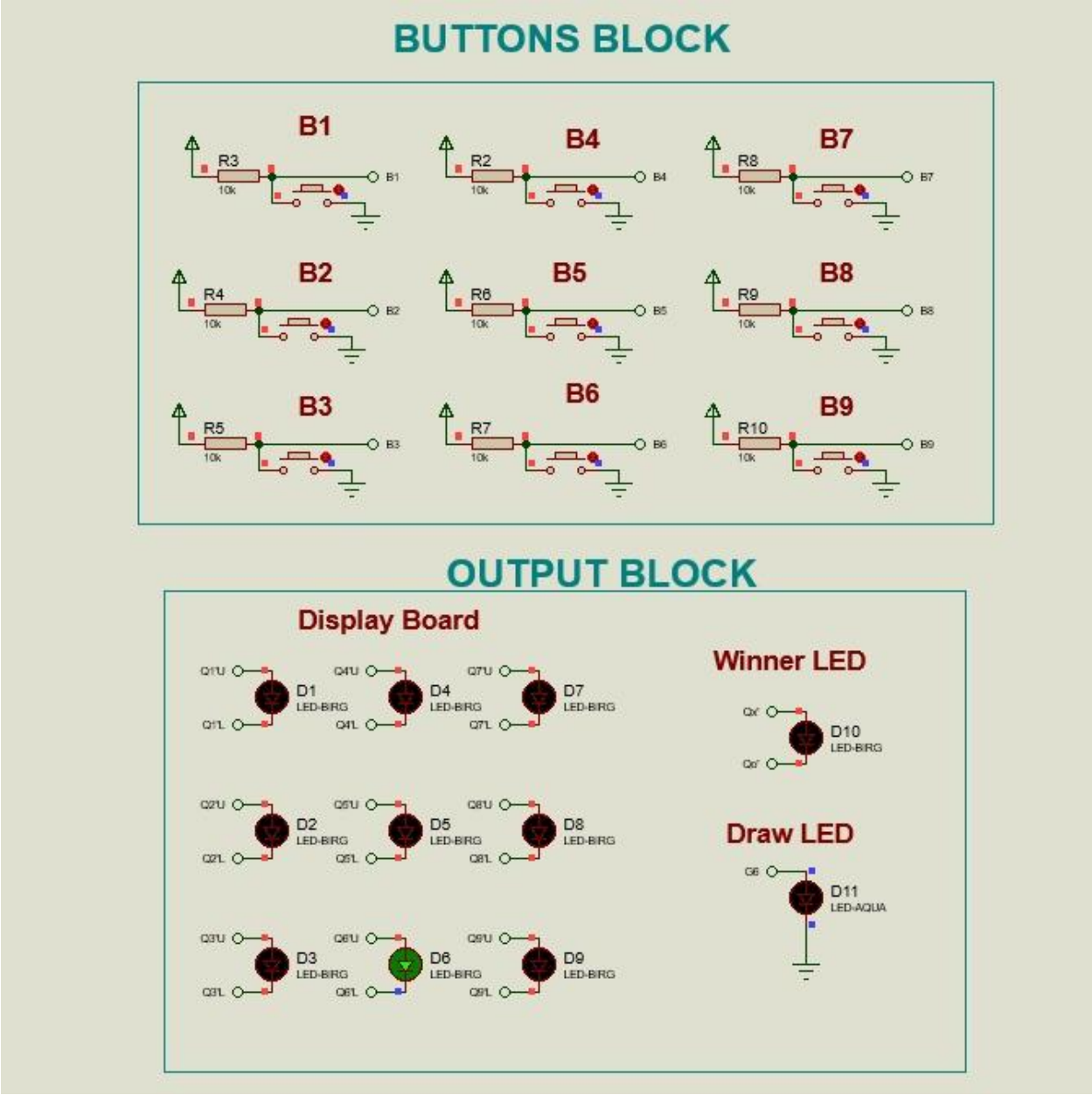
Error detention usage cases

Error check 1

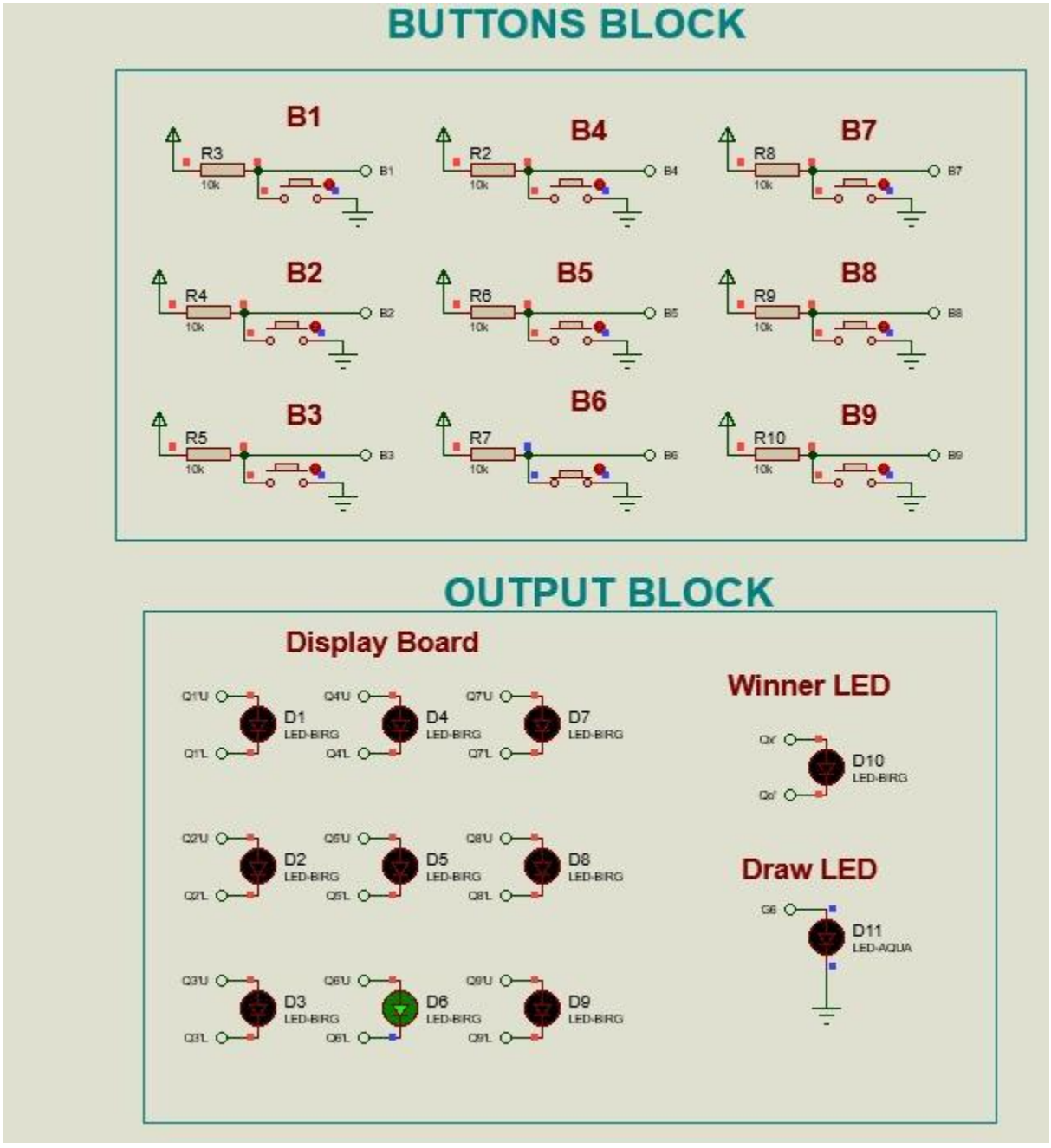
-
- No button Pressed



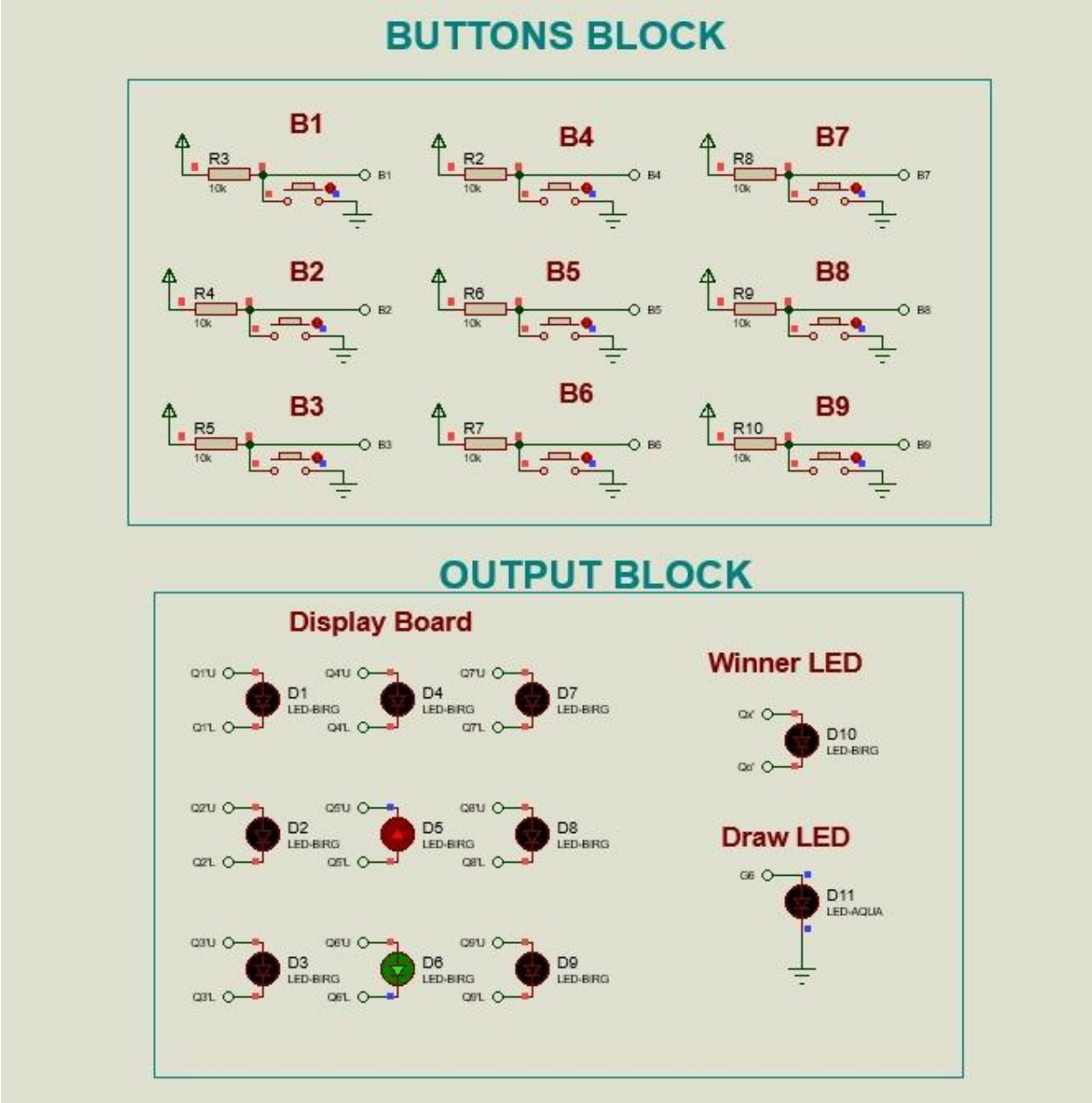
- Green Player presses B6



- Red Player tries to press B6.Led Display does not change

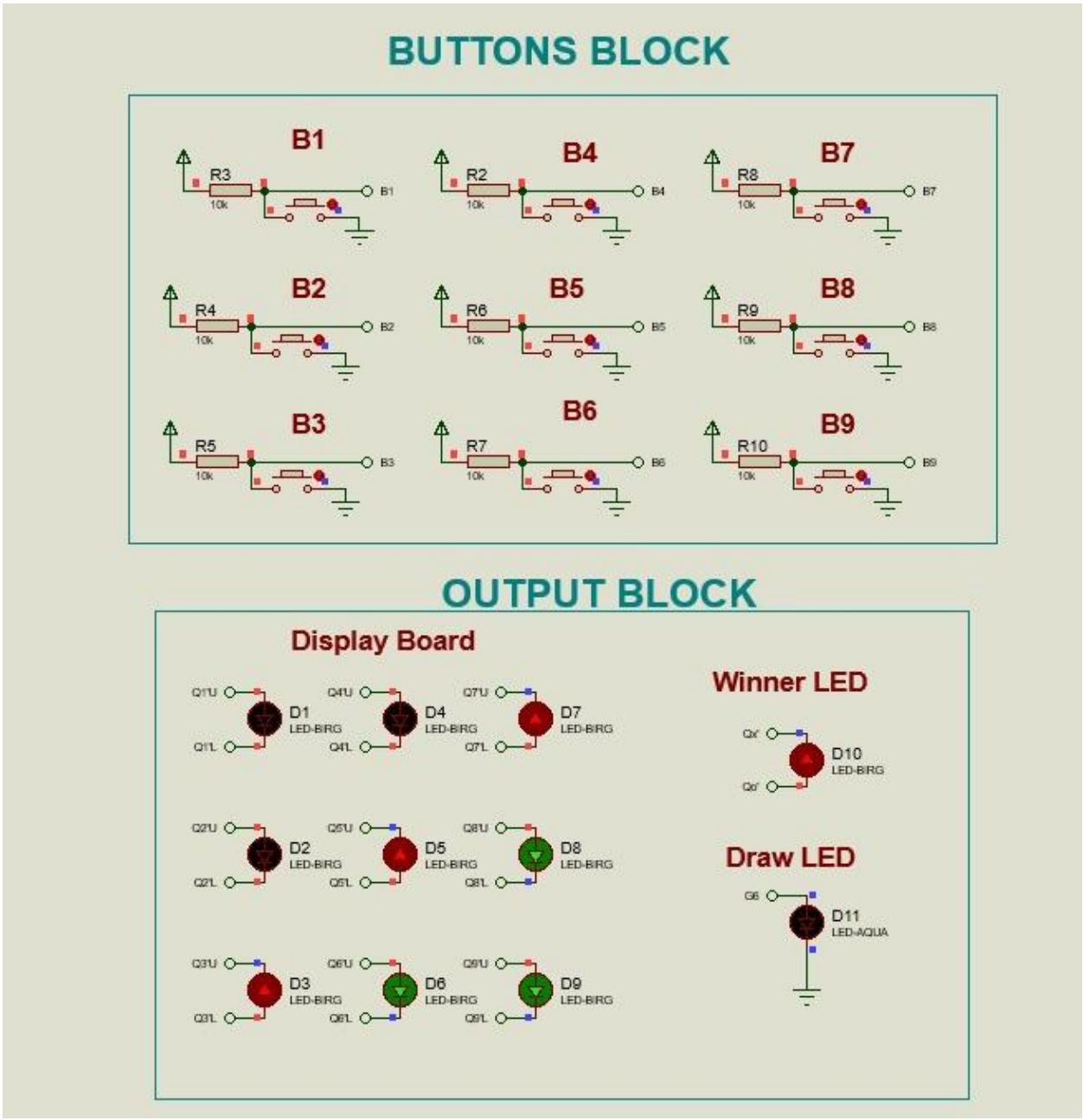


- Red player finally does their turn elsewhere. Their turn was not skipped.

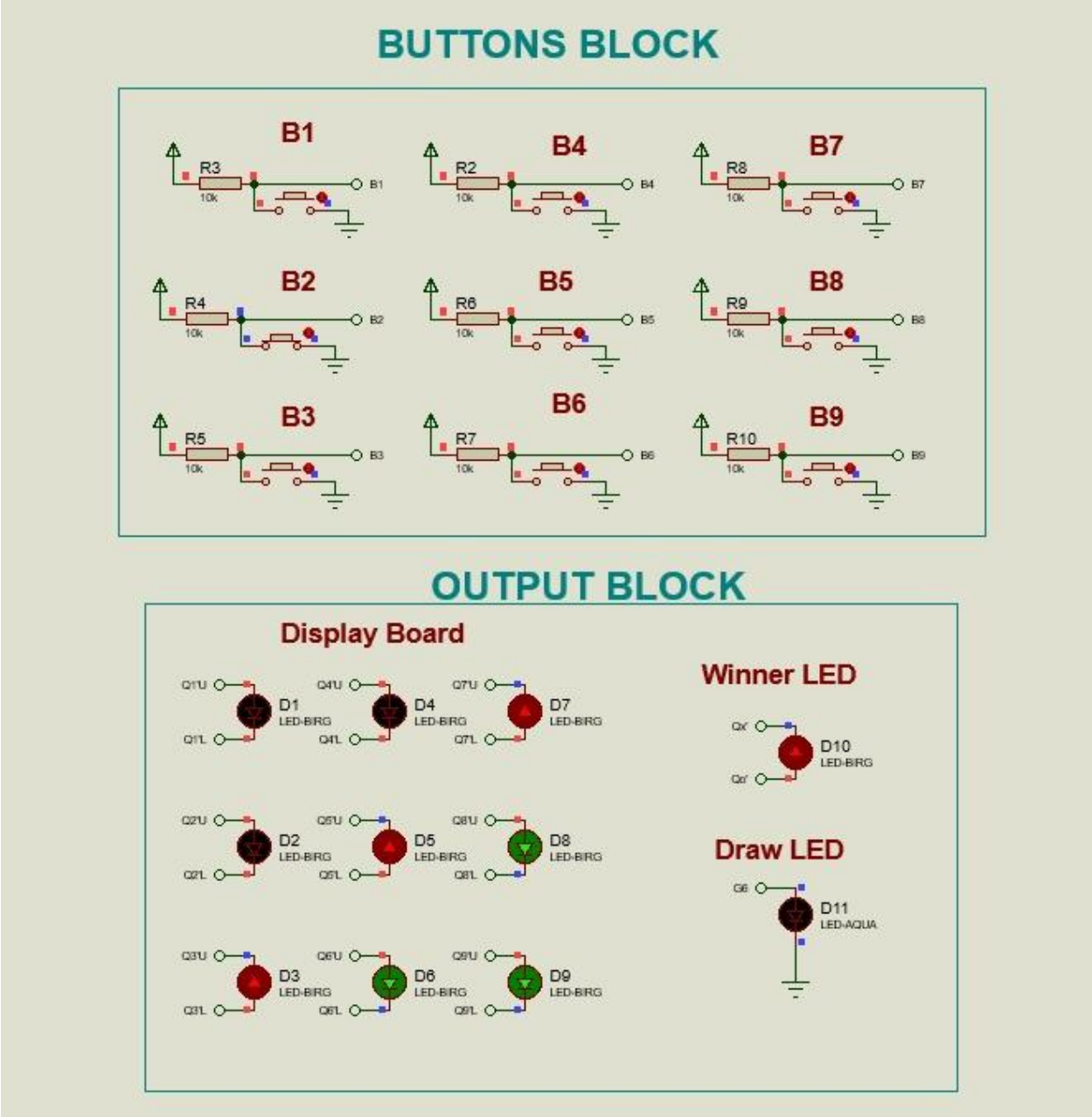


Error check 2

- Red Player won



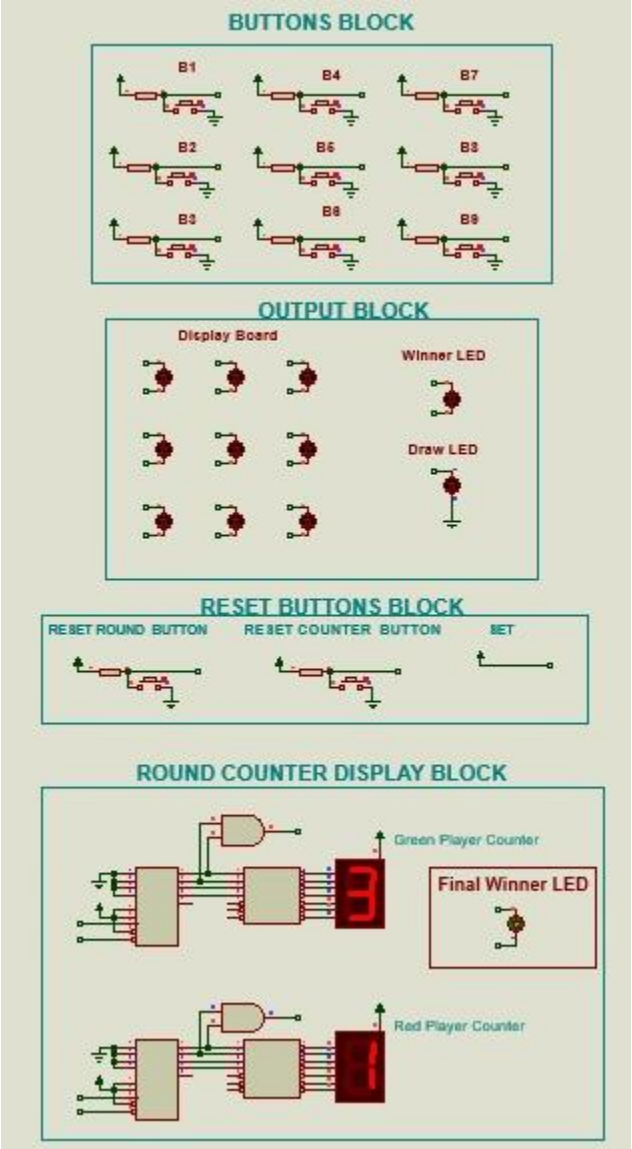
- Any player tries to play a button after round(B2 pressed here)



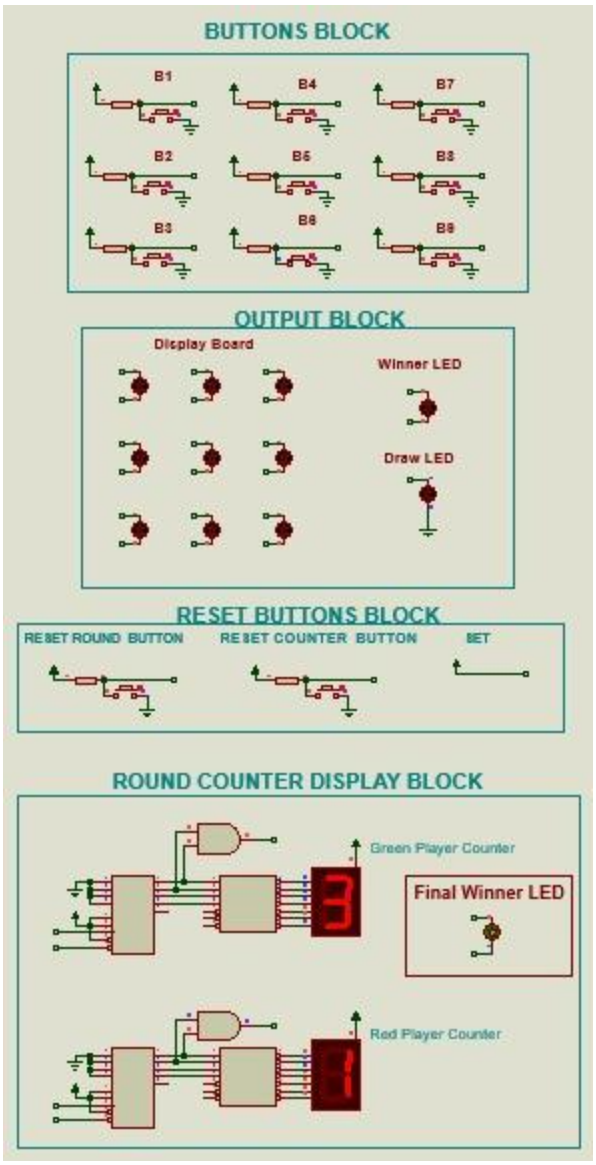
- Value not entered display stays the same

Error check 3

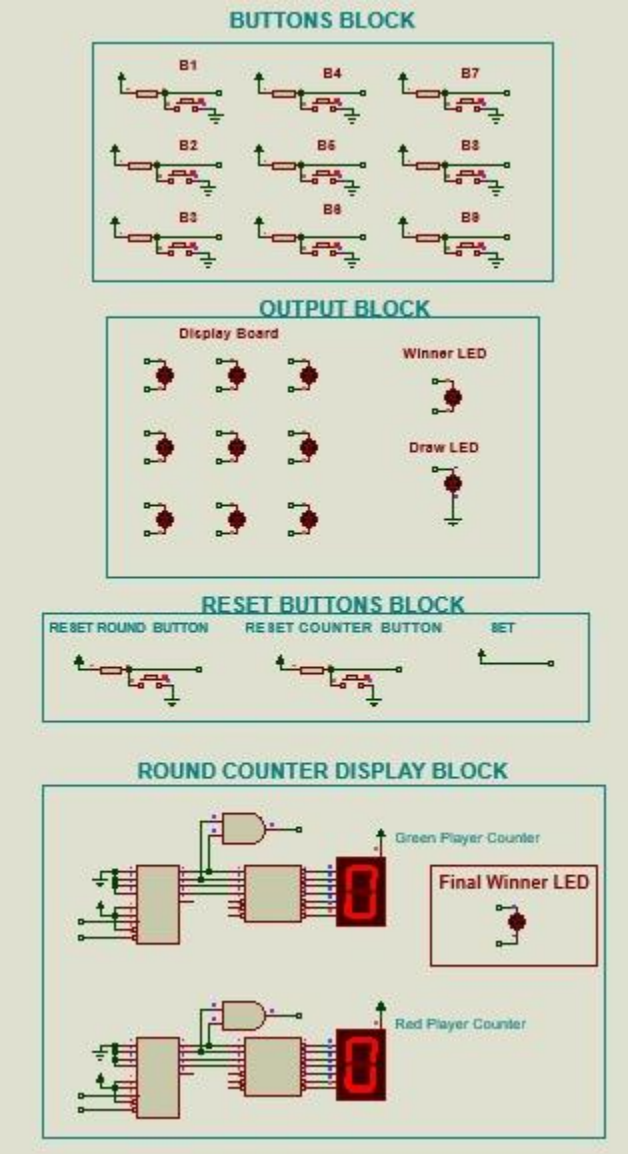
- Green player has won three round



- Any player tries to push a button to record another round on the counter. No inputs registered or shown on LED . Hence, the counters never move past 3. Even pressing reset round would not allow users to run another round after 3 rounds won by 1 of 2 players(B6 pressed here)


BUTTONS BLOCK
B1
B4
B7
B2
B5
B8
B3
B6
B9
OUTPUT BLOCK
Display Board
Winner LED
Draw LED
RESET BUTTONS BLOCK
RESET ROUND BUTTON
RESET COUNTER BUTTON
SET
ROUND COUNTER DISPLAY BLOCK
Green Player Counter
Final Winner LED
Red Player Counter

- Reset configuration can be applied.Now game can be played again



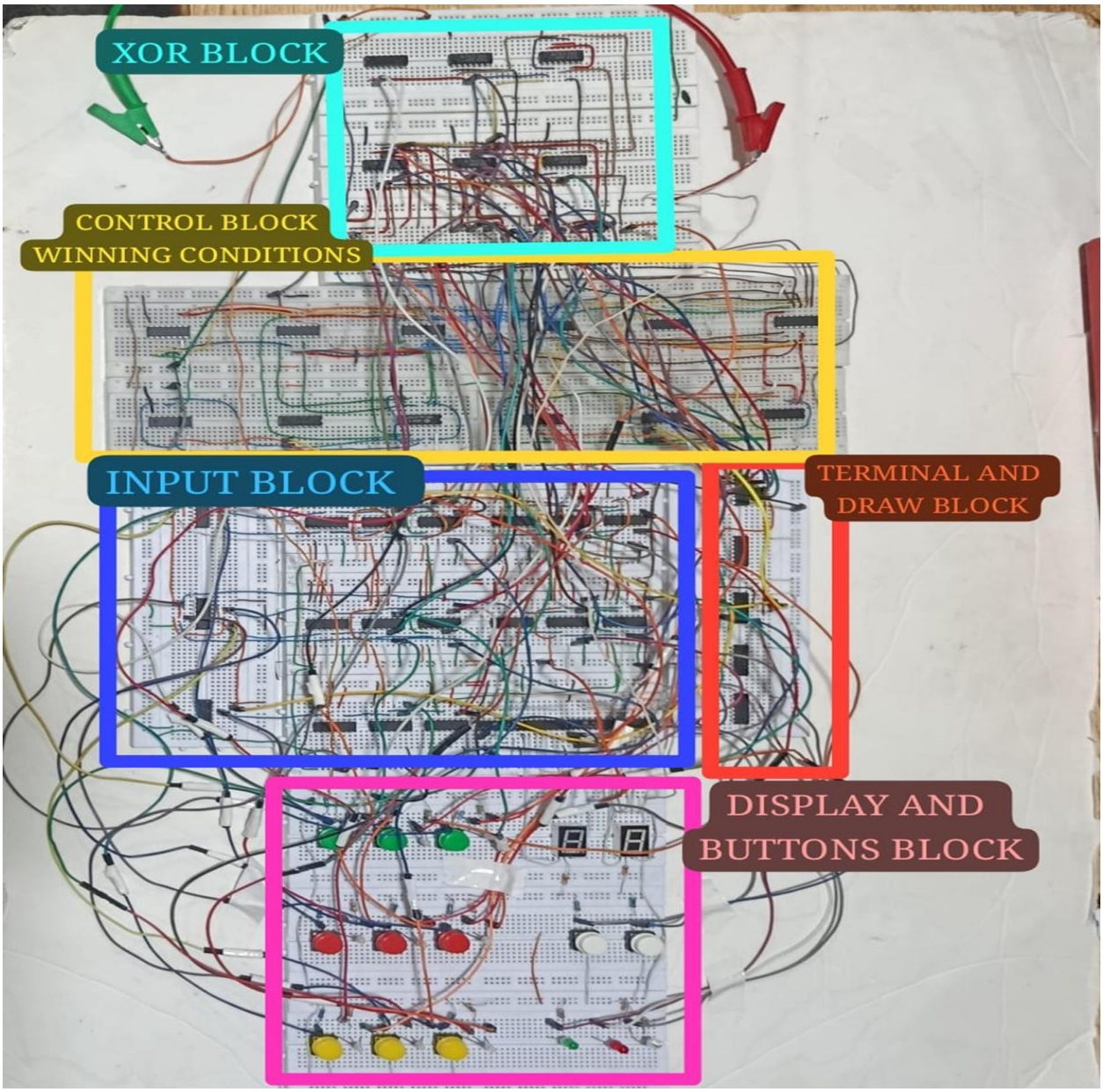
Error Detection

- No LED must be overridden
- No player after round is over or round winner LED is on or winning conditions fulfilled for one of 2 players can make next move
- No button pushes registered after a player has won i.e. after 1 of 2 counters reaches score of 3

Chapter 3: Hardware Implementation

3.1 Detailed Schematic of Design and its Description

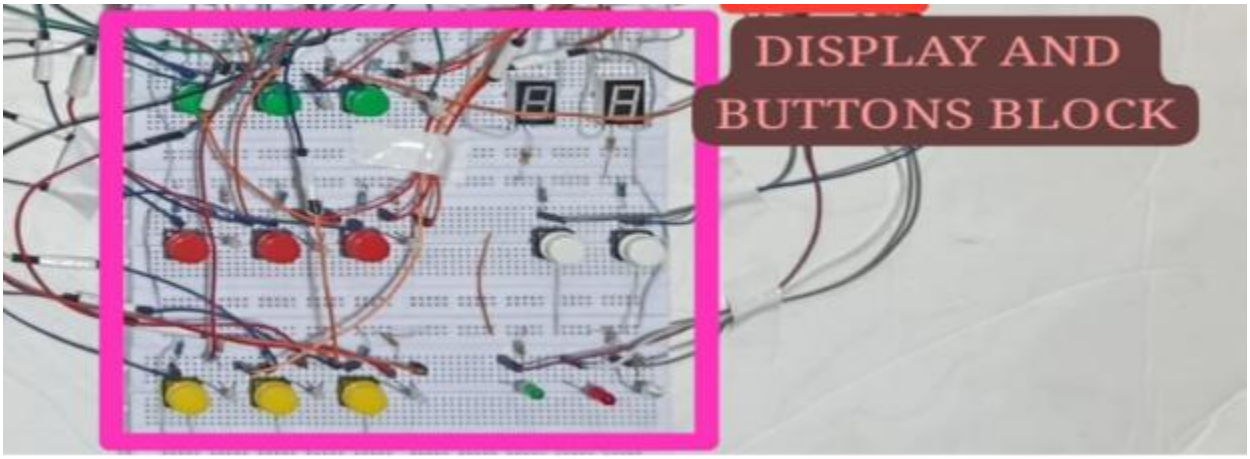
The hardware implementation of the Tic-Tac-Toe game follows a modular wiring scheme, with each functional block built and tested independently before integration. The design translates digital logic concepts directly into physical components, maintaining the same block-based architecture established during simulation.



The system is organized into the following hardware stages:

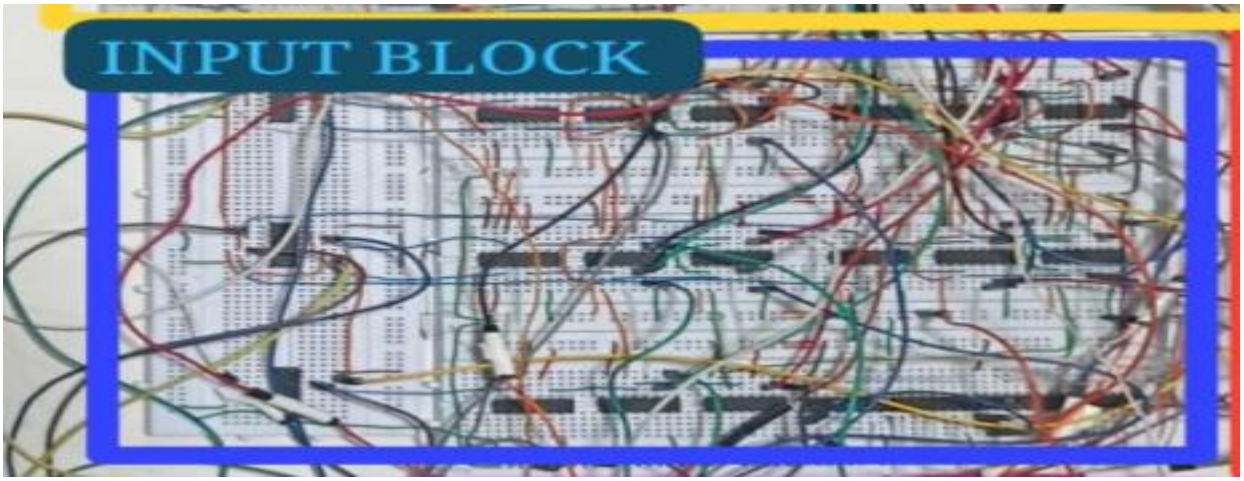
1. Input Stage

Nine push-button switches, numbered 1 through 9, correspond to the nine cells of the Tic-Tac-Toe grid. Each button press sends a signal into the Input Block. A separate reset switch allows the system to clear the entire board state and begin a new round. Pull-UP resistors (1 kΩ) are connected to each button input to ensure that the logic inputs remain at a stable Logic '0' when no button is pressed, preventing floating inputs that could cause unpredictable behaviour in the 74LS-series ICs.



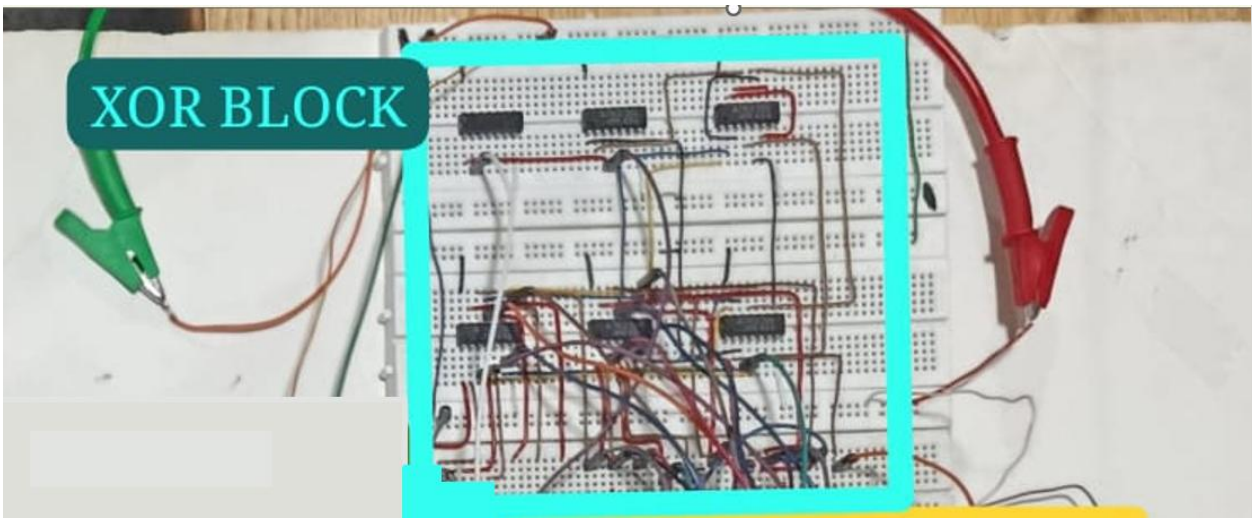
2. Processing Stage — Input Block (D Flip-Flops)

The core memory of the system is implemented using 74LS74 Dual D-type Flip-Flops. Each of the nine cells has a dedicated flip-flop that latches the state of that cell once a valid move is made. Once a cell is claimed by either Player 1 or Player 2, the flip-flop locks that state, ensuring no overwriting is possible for the remainder of the round. This enforces one of the most fundamental rules of the game at the **hardware** level.



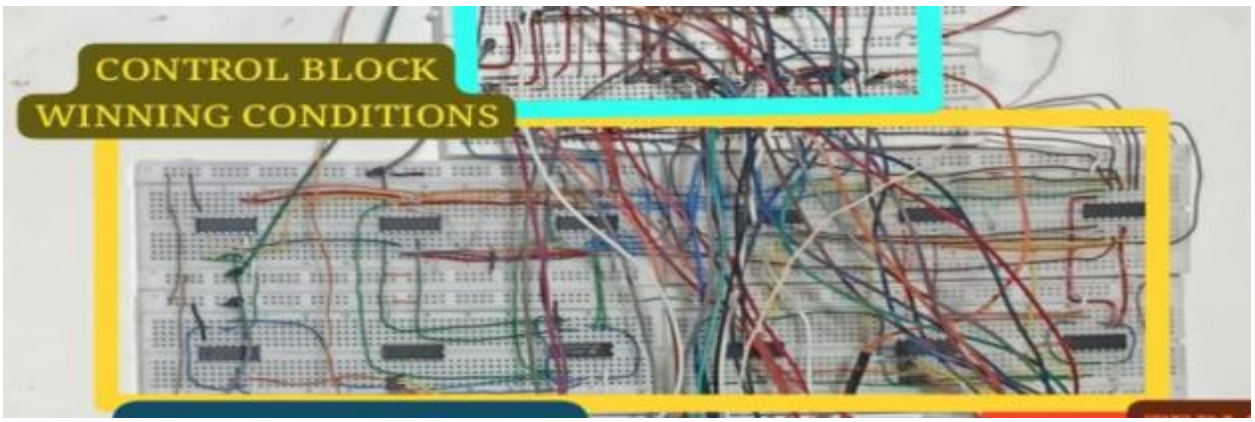
3. Processing Stage — XOR Block (Player Parity)

A 74LS86 Quad XOR Gate is used to implement player parity — the mechanism that alternates turns between Player 1 and Player 2. The XOR block feeds back into the flip-flop network: when Player 1's signal is high, the flip-flop locks Player 1's state for that cell; when Player 2's signal is high, it locks Player 2's state. This ensures that only one player can claim any given cell and that turns alternate correctly throughout the game.



4. Decision Stage — Control Block (Win Condition Logic)

The Control Block evaluates all eight possible winning conditions for each player — three rows, three columns, and two diagonals. This is implemented using 74LS11 3-input AND gates and 74LS08 Quad 2-input AND gates. For any winning line, all three relevant flip-flop outputs are fed into an AND gate; if all three are high for the same player, a win is declared. The outputs from all eight AND gates are then OR'd together using 74LS32 gates to produce a single Player 1 Win or Player 2 Win signal.



5. Decision Stage — Terminator Block (Draw & Game Over)

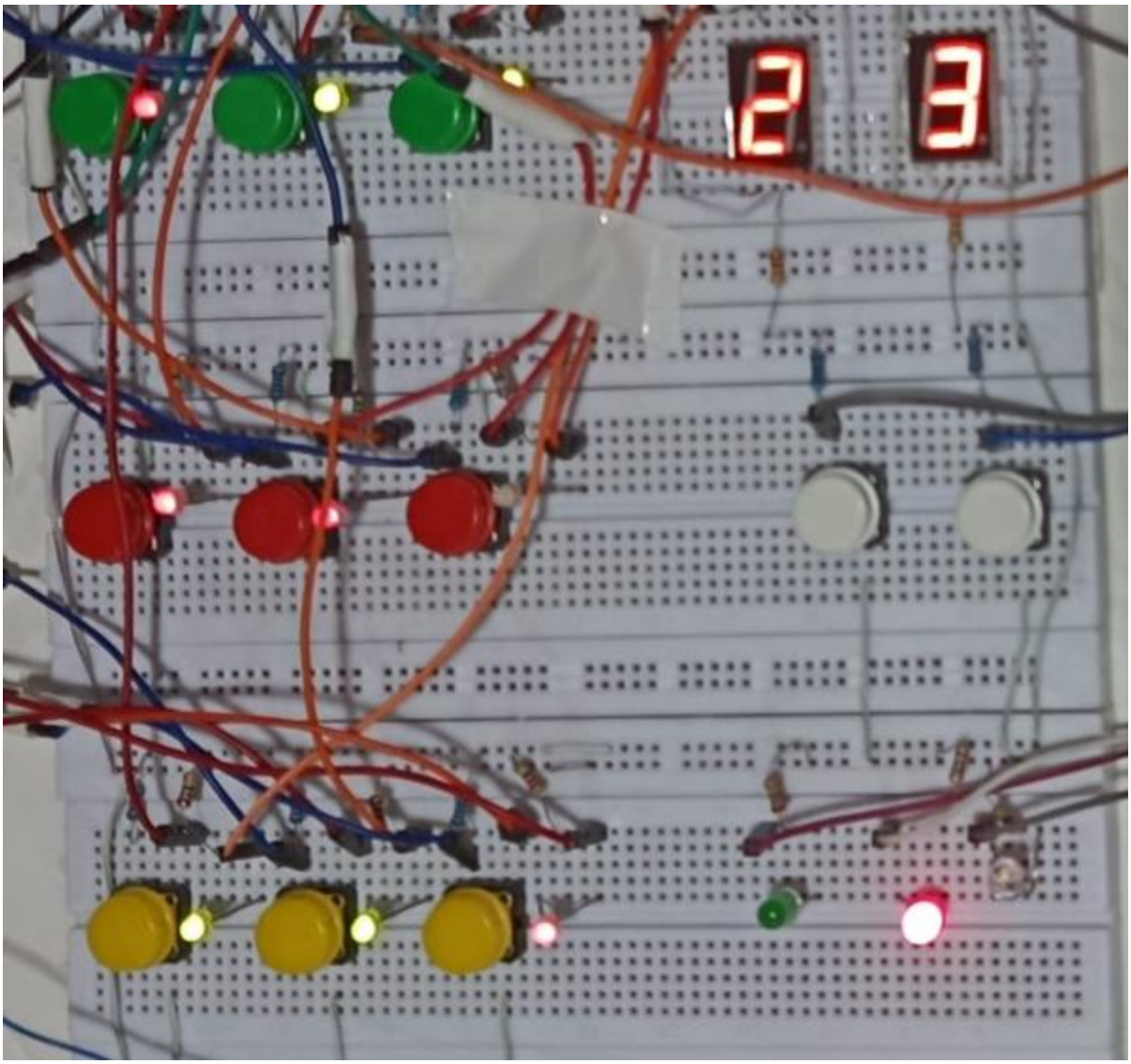
The Terminator Block determines draw conditions and handles overall game-over validation. A draw is detected when all nine cells are occupied (all nine flip-flops are latched) and neither player's win signal is active. The block also disables further input once a win or draw has been detected, preventing additional button presses from affecting the result.

6. Counter Stage

A 74LS161 Synchronous 4-bit Counter is used to track each player's score across multiple rounds. The counter increments each time the corresponding player wins a round. The score is decoded and displayed on a 7-Segment Display using the SN7447A BCD-to-7-Segment Decoder, allowing both players to see cumulative scores at all times.

7. Output Stage — LED Grid Display

The 3×3 LED grid directly mirrors the button layout, providing real-time visual feedback of the board state. Each LED corresponds to one of the nine cells and is driven by the output of its respective flip-flop. Separate indicator LEDs signal the result of each round — Player 1 Win, Player 2 Win, or Draw. Current-limiting resistors (330 Ω) are placed in series with every LED to protect the output stages of the ICs from over-current damage and to ensure uniform brightness across all indicators.



3.2 Details of ICs used

The following integrated circuits were used in the hardware implementation:

- **74LS74** — Dual D-type Flip-Flop with Preset and Clear — Used as cell memory for all nine grid positions and player parity storage.
- **74LS08** — Quad 2-Input AND Gate — Used in win-condition logic for rows, columns, and diagonals.
- **74LS11** — Triple 3-Input AND Gate — Used for evaluating 3-cell winning combinations.
- **74LS32** — Quad 2-Input OR Gate — Used to combine outputs of all eight win conditions into a single win signal per player.
- **74LS86** — Quad 2-Input XOR Gate — Used for player parity (turn alternation) logic.
- **74LS04** — Hex Inverter — Used for signal inversion where required in control and input validation logic.
- **74LS161** — Synchronous 4-bit Counter — Used to count and store round wins for each player.

- **SN7447A** — BCD to 7-Segment Decoder — Used to drive the 7-segment displays for score output.

3.3 Details of Other Components used like diodes, transistors, resistors etc.

1. Resistors

- **1 kΩ Resistors (Pull-Up):** Connected to each push-button input to hold the logic line at a stable Logic '1' when the button is open/unpressed, preventing floating inputs and erratic behaviour.
- **330 Ω Resistors (Current Limiting):** Placed in series with each output LED and the win/draw indicator LEDs to limit current to a safe operating level (~15 mA at 5 V), protecting the IC output stages from damage.

2. Push-Button Switches

Nine momentary push-button switches represent the nine cells of the Tic-Tac-Toe grid. One additional button serves as the reset switch to clear the board state at the end of a round. All switches are debounced through the pull-down resistor network to minimize spurious triggering.

3. Output Indicators (LEDs)

- **3×3 LED Grid:** Nine LEDs arranged in a 3×3 pattern mirror the game grid, providing real-time visual representation of the board state.
- **Result LEDs:** Separate LEDs indicate Player 1 Win, Player 2 Win, and Draw outcomes.
- **7-Segment Displays:** Two 7-segment displays show the cumulative score for each player across rounds, driven by the SN7447A decoder.

3.4 Hardware Issues / Results/ Observations

Hardware Issues Encountered

1. Switch Bounce / Spurious Input Registration

- *Issue:* Mechanical push-button switches do not transition cleanly between states; they "bounce," generating multiple rapid transitions that the flip-flops registered as several presses instead of one. This caused cells to be claimed incorrectly or player turns to skip.
- *Solution:* Pull-up resistors at the inputs helped suppress noise. Additionally, the inherent propagation delay of the flip-flop network provided a natural filtering effect. Where necessary, the clock timing was adjusted to ensure only one stable rising edge was captured per press.

2. Loose Connections on the Breadboard

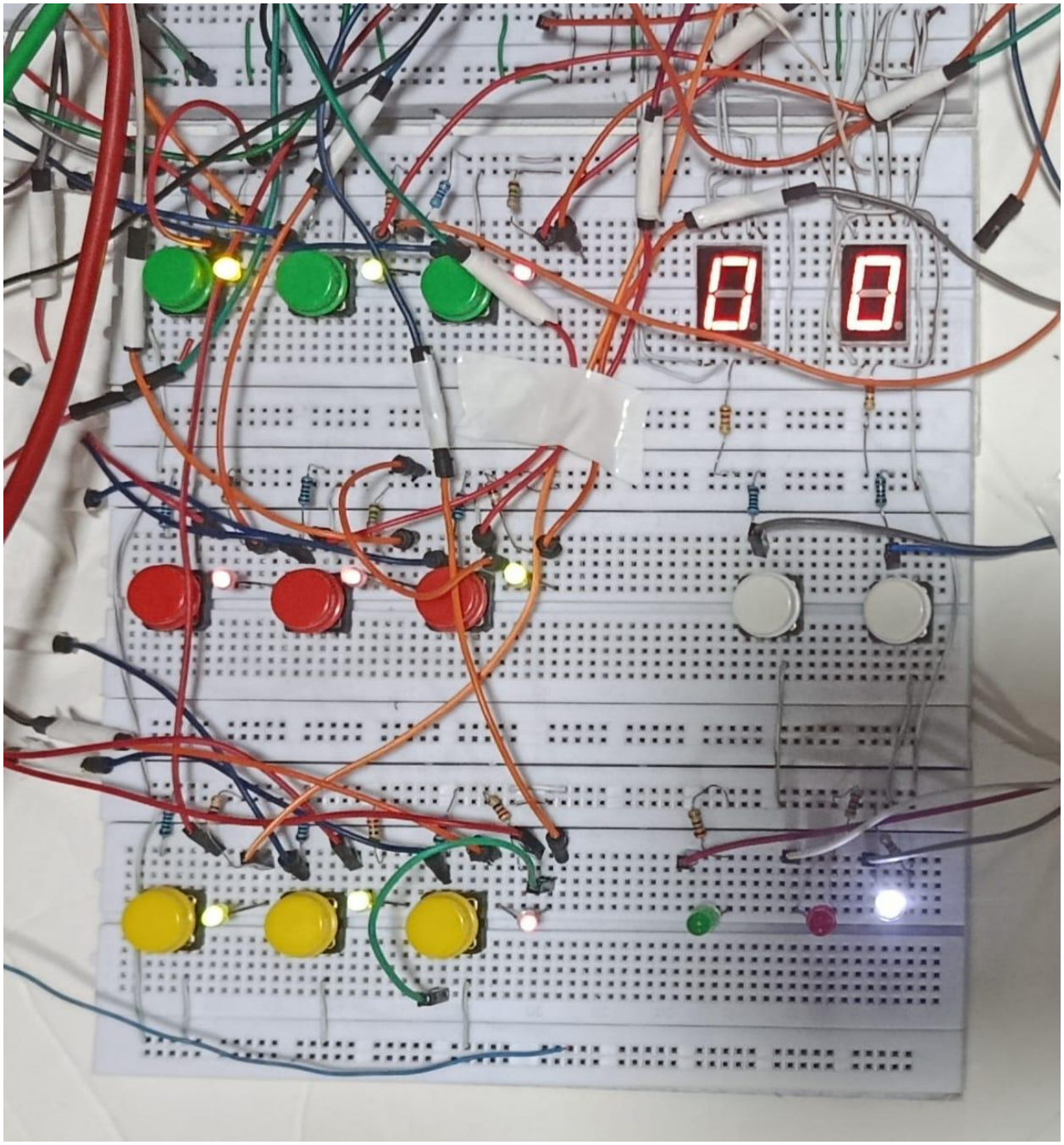
- *Issue:* The large number of interconnecting wires required for nine cells, eight win conditions, and the counter block led to frequent loose connections. This caused flickering LEDs and incorrect game states during testing.
- *Solution:* Continuity tests were performed using a multimeter to identify faulty connections. Shorter, rigid jumper wires were used for critical signal paths such as the flip-flop clock and win-condition outputs to minimize accidental disconnection.

Results & Observations

After resolving the above issues, the system was tested against all expected game scenarios:

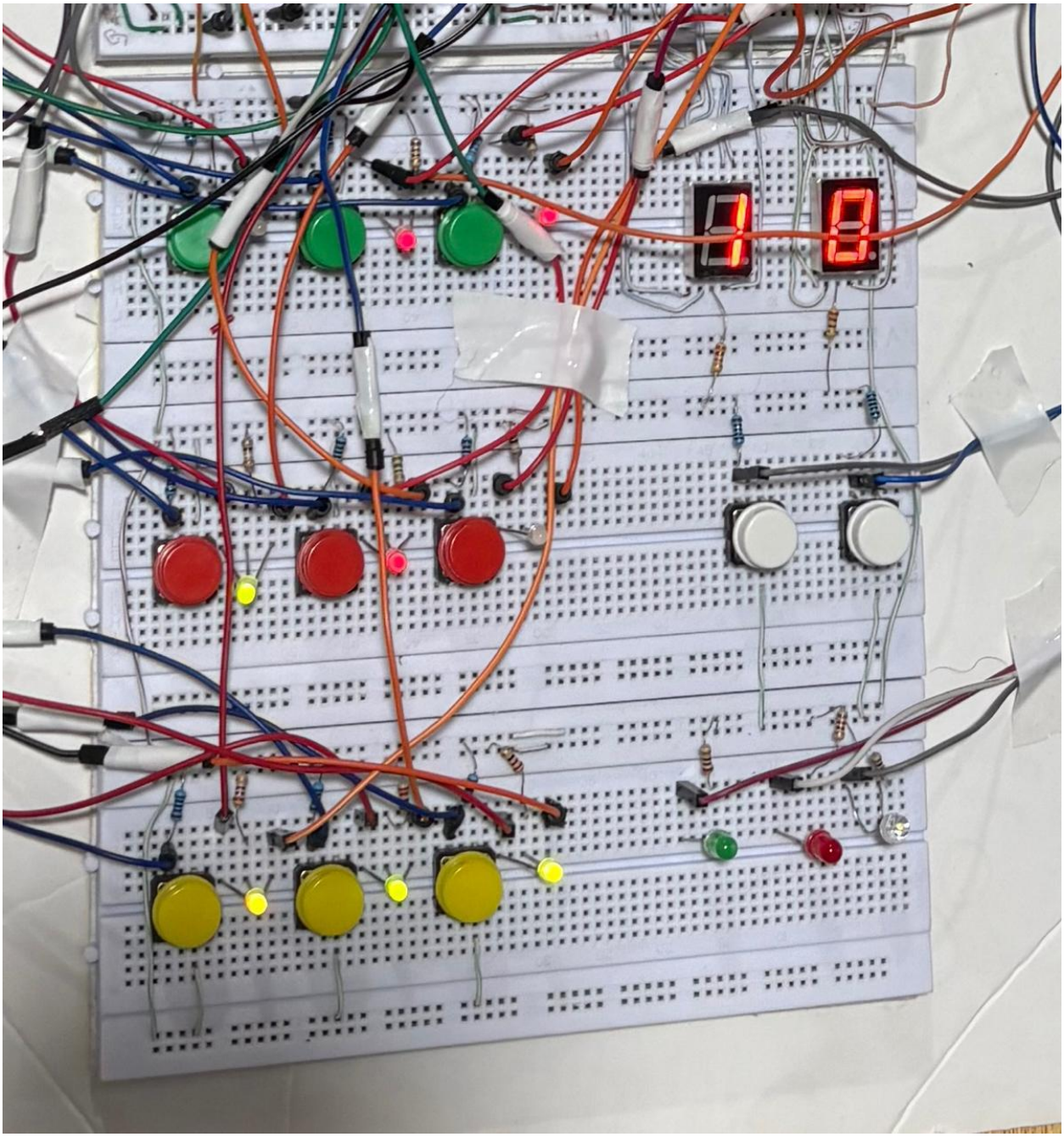
1. *Cell Locking:* Once a cell was claimed by either player, the flip-flop successfully prevented any subsequent input from overwriting that cell's state, even when the corresponding button was pressed repeatedly.
2. *Turn Alternation:* The XOR-based player parity logic correctly alternated control between Player 1 and Player 2 on every valid move, with no instances of the same player moving twice in a row during testing.
3. *Win Detection:* All eight winning conditions — three rows, three columns, and two diagonals — were verified for both players. The win signal triggered correctly in every tested scenario, and the corresponding result LED illuminated immediately.
4. *Draw Detection:* The draw condition was correctly identified when all nine cells were occupied and no winning condition was satisfied.
5. *Score Counting:* The 74LS161 counter incremented accurately after each round, and the SN7447A decoder displayed the correct score on the 7-segment display throughout multi-round testing.
6. *General Observation:* The decoupling capacitor visibly stabilized LED output during switching events, eliminating the intermittent flickering observed in early tests. The modular wiring approach allowed individual blocks to be isolated and tested independently, which significantly reduced debugging time.

DRAW:

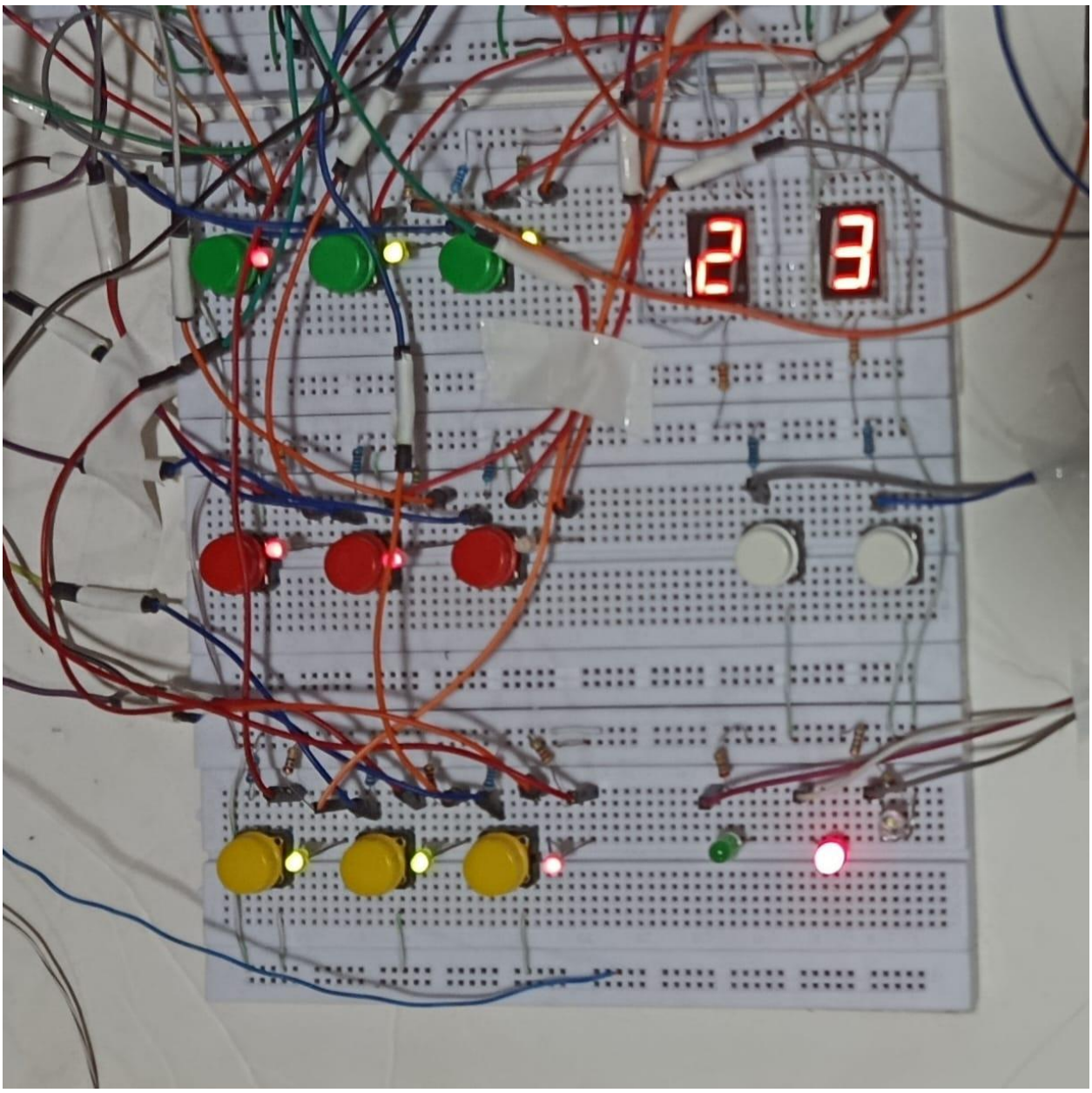


PLAYER 1 WINS:

Note : For round winners only counter goes up.



THREE ROUND WINNER:



Chapter 4: Project Applications

4.1 Core Applications

The digital logic concepts demonstrated in this Tic-Tac-Toe project extend well beyond a simple game. The underlying architecture — combining sequential memory, combinational decision logic, turn management, and score tracking — maps directly onto real-world digital system design:

1. Sequential Logic & Embedded State Machines

The use of D flip-flops to store and lock cell states is identical to the design of state machines used in embedded systems. Vending machines, traffic light controllers, elevator control systems, and access control panels all rely on the same principle: a system that remembers its current state, accepts an input, and transitions to a new state based on defined logic. This project provides hands-on experience with the physical building block of all such systems.

2. Human-Machine Interface (HMI) Design

The project demonstrates a complete input-processing-output pipeline — push buttons accept user input, logic circuits process it, and LEDs communicate the result. This is the fundamental model for any human-machine interface, from industrial control panels to consumer electronics. The design principle of providing immediate visual feedback (cell LEDs updating on each move) reflects best practices in real-world HMI systems.

3. Digital Game Logic & Consumer Electronics

The Boolean win-detection logic implemented here is directly applicable to game console hardware design. Early gaming systems like the Atari 2600 implemented game rules entirely in combinational and sequential logic, identical to the approach used in this project. The architecture also applies to any decision-rule system where a set of conditions must be monitored simultaneously and an action triggered when a combination is satisfied.

4. Score Tracking & Event Counting Systems

The 74LS161 counter combined with the BCD-to-7-Segment decoder forms a complete event-counting and display subsystem. This configuration is used in digital scoreboards, industrial production counters, frequency meters, and sports timing systems. The project demonstrates how a general-purpose synchronous counter can be adapted to count meaningful application-specific events.

5. Educational Tool for Digital Logic Concepts

This project serves as a tangible, interactive demonstration of core Digital Logic Design concepts including flip-flops, combinational logic, Boolean simplification, cascading, and modular design. Its value as a teaching tool is significant — abstract concepts like state storage and win-condition evaluation

become immediately intuitive when a student can physically press a button and observe the logic respond in real time.

Chapter 5: Future Recommendations

5.1 Proposed Improvements

While the current design successfully implements a fully functional two-player Tic-Tac-Toe game using discrete digital logic ICs, several enhancements could be made in future iterations to increase functionality, usability, and performance:

1. Player Move Countdown Timer

The current system has no time restriction on how long a player may take to make a move. A future improvement would be to integrate a countdown timer using an additional 74LS161 counter configured to count down from a preset value. When the timer reaches zero, the turn would automatically forfeit or trigger a penalty signal. This would add a competitive element and more closely mimic real-time game environments. The timer output could be displayed on an additional 7-segment display alongside the existing score displays.

2. Audio Feedback Using Buzzers

Currently, all game feedback is visual — LEDs indicate cell ownership and win/draw outcomes. Integrating a piezoelectric buzzer driven by the win, draw, and invalid-move signals would provide audio confirmation of game events. Different tones could be generated for a valid move, an invalid move attempt, a win, and a draw, significantly improving the user experience and making the game more intuitive to play without requiring the user to constantly monitor the LED indicators.

3. Generalised Architecture for Larger Grids (4×4 Tic-Tac-Toe)

The current architecture is designed specifically for a 3×3 grid with eight winning conditions. Scaling the design to a 4×4 grid would require sixteen cell flip-flops and ten winning conditions (four rows, four columns, and two diagonals). While this significantly increases the IC count and wiring complexity, the modular block-based design of the current system makes such an expansion structurally straightforward. Larger grids would also increase the strategic depth of the game and serve as a more complex demonstration of scalable digital logic design.

4. Single-Player Mode Against a Hardwired Opponent

The current implementation requires two human players. A compelling future addition would be a single-player mode in which one player competes against a hardwired logic-based opponent. A basic strategy could be implemented using combinational logic that checks for winning opportunities and blocking moves in priority order — for example, completing a winning line if two of the opponent's cells are already filled, or blocking the human player's potential win. While a fully optimal AI would require a microcontroller, a reasonable rule-based opponent is achievable entirely in hardware using additional AND, OR, and priority logic.

5. Printed Circuit Board (PCB) Implementation

The breadboard implementation, while excellent for prototyping and debugging, is inherently prone to loose connections, signal noise, and wiring clutter — as observed during the hardware testing phase of this project. Transferring the finalized design onto a custom Printed Circuit Board (PCB) would eliminate parasitic capacitance from long jumper wires, ensure permanent and reliable connections, reduce the physical footprint of the circuit, and allow the system to operate at higher switching speeds with greater noise immunity. A PCB implementation would also present the project in a polished, professional form factor suitable for demonstration and further development.

6. Non-Volatile Score Memory

The current score-tracking system using the 74LS161 counter loses all data when power is removed, as the counter resets to zero on startup. Integrating a simple non-volatile memory element — such as a battery-backed latch or a small EEPROM — would allow player scores to persist across power cycles, making the system suitable for longer tournaments without requiring continuous power.

References / Bibliography

Datasheets

1. **Texas Instruments, "Dual D-Type Positive-Edge-Triggered Flip-Flops with Preset and Clear (SN74LS74A)." Available: SN74LS74A Datasheet.**
2. **Texas Instruments, "Quadruple 2-Input Positive-AND Gates (SN74LS08)." Available: SN74LS08 Datasheet.**
3. **Texas Instruments, "Quadruple 2-Input Positive-OR Gates (SN74LS32)." Available: SN74LS32 Datasheet.**
4. **Texas Instruments, "Quadruple 2-Input Exclusive-OR Gates (SN74LS86)." Available: SN74LS86 Datasheet.**
5. **Texas Instruments, "Synchronous 4-Bit Binary Counters (SN74LS161A)." Available: SN74LS161A Datasheet.**
6. **Texas Instruments, "BCD-to-Seven-Segment Decoders/Drivers (SN7447A)." Available: SN7447A Datasheet.**
7. **Texas Instruments, "Triple 3-Input Positive-AND Gates (SN74LS11)." Available: SN74LS11 Datasheet.**
8. **Texas Instruments, "Hex Inverters (SN74LS04)." Available: SN74LS04 Datasheet.**

Software Tools

9. **Labcenter Electronics Ltd., *Proteus Design Suite*, Simulation Software. Available: Official Site.**

Textbooks

10. **M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5th ed. Pearson Education, 2013.**
11. **John F. Wakerly, *Digital Design: Principles and Practices*, 4th ed. Pearson Prentice Hall, 2006.**

Online References

12. **All About Circuits, "Digital Logic Design Resources." Available: https://www.allaboutcircuits.com**
13. **Electronics Tutorials, "D-type Flip Flop Circuit." Available: https://www.electronics-tutorials.ws**